

0.2 Arquitectura RISC-V

ASM

Versión 1.0.2

0.2.0 RISC-V

Arquitectura de programación de una computadora (ISA)

Modos de
direccionamiento

Set de
instrucciones

Memoria
direccionable

Registros
accesible

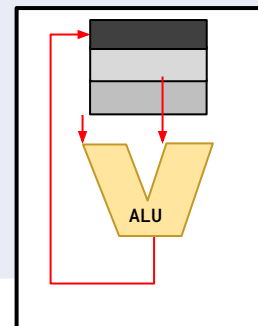
La arquitectura RISC-V se define como una arquitectura base operando sobre números enteros, los cuales pueden ser de **32 bits (RV32i) o 64 bits (RV64i)**. El conjunto de operaciones mínimo RV32i define operaciones aritméticas (suma, resta, desplazamientos) y lógicas (and, or, xor). Existen extensiones que agregan operaciones como multiplicación y división, extensiones que permiten representar instrucciones de forma comprimida y extensiones que permiten resolver ciertas operaciones de forma atómica (sin paralelismo). Este conjunto de extensiones es el más común y se lo conoce como RV32imac.

En este curso nos limitamos a trabajar con el mínimo indispensable que es RV32i.

El tipo de arquitectura es similar al tipo Registro-Registro, donde en cada instrucción interviene al menos un registro.

Registro-registro

```
load R1,A  
load R2,B  
add R3,R1,R2  
store C,R3
```



Memoria Direccional: Al ser una arquitectura Harvard, existe una memoria de instrucciones y una memoria de datos. El espacio de direccionamiento es el mismo para ambos existiendo **32 bits para direcciones**. Esto quiere decir que se pueden direccionar **2^{32} direcciones** de un byte, es decir **4GB**. Dado que **la palabra es de 32 bits**, también podemos decir que **direcciona 1G Palabras**. El Program Counter (PC) es utilizado para acceder a instrucciones. Teniendo en cuenta que **direcciona al byte**, cada instrucción consume 4 direcciones consecutivas. Es por esto que se incrementa de a 4 entre instrucciones. La memoria de instrucciones y la memoria de datos pueden ser físicamente la misma siempre y cuando posean puertos distintos para cada fin. La entrada y la salida de datos de memoria se encuentran separadas.

Esto evita tener que estar cambiando de sentido en el bus de datos.

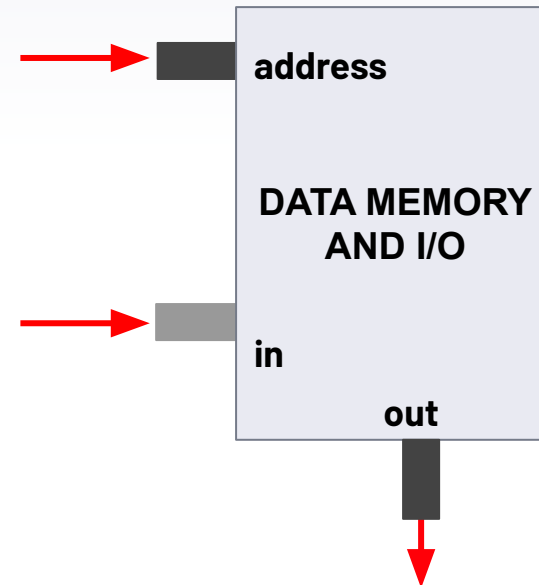
Arquitectura de programación de una computadora (ISA)

Modos de
direccionamiento

Set de
instrucciones

Memoria
direccionable

Registros
accesible



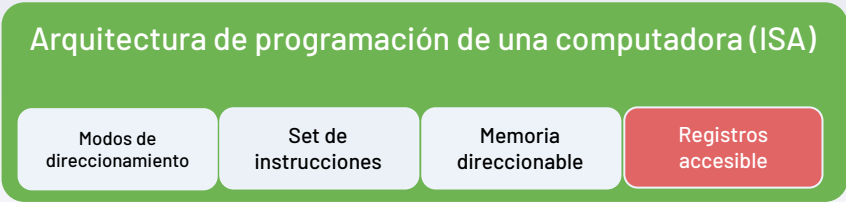
Registros accesibles: La arquitectura posee 1 registro **Program Counter** y **32 registros de propósitos generales**. De esos 32 registros, el registro cero (**zero, x0**) siempre contiene el valor 0. Si almacenamos un valor en x0 el mismo se descarta. Si leemos el registro x0 siempre obtenemos 0. El resto de los registros pueden utilizarse libremente. Existe una convención dentro de la arquitectura sobre cuál es el uso que debería darse a los mismos. El ensamblador soporta referencias por número de registro:

Ejemplo: x0, x1, x2, x31

Aunque también soporta referencias por nombre

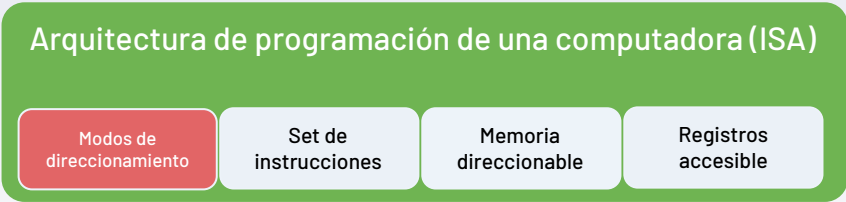
Ejemplo: zero, ra , a0, s11.

El PC no aparece en la lista ya que solo podemos interactuar con él mediante ciertas instrucciones específicas (por ejemplo saltos). Sin embargo es siempre un registro accesible por el programador.



Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5	t0	Temporary/alternate link register	Caller
x6–7	t1–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller

Modos de direccionamiento: Si bien ya vimos ejemplos de modos de direccionamiento en RISC-V, existen 6 formatos de instrucción definidos en RV32i.



31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Todas las instrucciones se representan con 32 bits. Los **7 bits menos significativos** indican el **código de operación**. Luego en operaciones aritmético-lógicas existen los bits de **funct3 y funct7** para definir la **operación en la ALU**. Esto es típico de unidades de control cableadas. Las referencias a registros de operandos son **rs1 y rs2** y el registro destino es siempre **rd**. Distintos formatos codifican los inmediatos (constantes) de diversas formas. Los modos de direccionamiento incluyen: Modo registro directo, Modo base más desplazamiento y modo inmediato.

Set de Instrucciones: El conjunto RV32i posee 47 instrucciones. Existen 9 que no utilizaremos (CSR*, Fence*, Break) en principio. Del resto:

Arquitectura de programación de una computadora (ISA)

Modos de
direccionamiento

Set de
instrucciones

Memoria
direccionable

Registros
accesible

- **Aritméticas:** ADD, SUB, SLL, SRL, SRA, SLT (y sus versiones inmediatas ADDI, SLLI, SRLI, SRAI, SLTI)
- **Lógicas:** AND, OR, XOR (y sus versiones inmediatas ANDI, ORI, XORI)
- **Comparación y salto:** BEQ, BNE, BLT, BGE (y sus versiones unsigned BLTU, BGEU)
- **Lectura de memoria:** LW, LH, LB (y sus versiones unsigned LHU, LBU)
- **Escritura de memoria:** SW, SH, SB
- **Salto relativo al PC:** JAL
- **Salto relativo a Registro:** JALR
- **Asignación a registro con inmediato:** LUI
- **Asignación a registro con PC e inmediato:** AUIPC

Al ser una arquitectura RISC, muchas operaciones consumen varias instrucciones. Dado que se repiten ciertos conjuntos de instrucciones existen pseudo-instrucciones que representan operaciones complejas: LI (Load Immediate), LA (Load Address), RET (Return), NOP (No operation), NOT (complemento a la base menos uno), NEG (complemento a la base). El ensamblador transforma estas pseudo-instrucciones en una o más de las 47 instrucciones reales según el caso.

Nota: Los números se asumen signados salvo que la instrucción especifique unsigned. Todas las constantes (immediate) se expanden a 32 bits con su signo propagado.

Arquitectura de programación de una computadora (ISA)

Modos de
direccionamiento

Set de
instrucciones

Memoria
direccionable

Registros
accesible

Ej: addi x2,x3,-1 utiliza 12 bits para codificar -1, pero el número final utiliza 32 bits.

- **Aritméticas:** ADD , SUB , SLL , SRL, SRA, SLT (y sus versiones inmediatas ADDI, SLLI, SRLI, SRAI, SLTI)
- **Lógicas:** AND, OR, XOR (y sus versiones inmediatas ANDI, ORI, XORI)
- **Comparación y salto:** BEQ, BNE, BLT, BGE (y sus versiones unsigned BLTU, BGEU)
- **Lectura de memoria:** LW, LH, LB (y sus versiones unsigned LHU, LBU)
- **Escritura de memoria:** SW, SH, SB
- **Salto relativo al PC :** JAL
- **Salto relativo a Registro:** JALR
- **Asignación a registro con inmediato:** LUI
- **Asignación a registro con PC e inmediato:** AUIPC

Al ser una arquitectura RISC, muchas operaciones consumen varias instrucciones. Dado que se repiten ciertos conjuntos de instrucciones existen pseudo-instrucciones que representan operaciones complejas:

LI (Load Immediate), LA (Load Address), RET (Return), NOP (No operation), NOT (complemento a la base menos uno), NEG (complemento a la base). El ensamblador transforma estas pseudo-instrucciones en una o más de las 47 instrucciones reales según el caso.



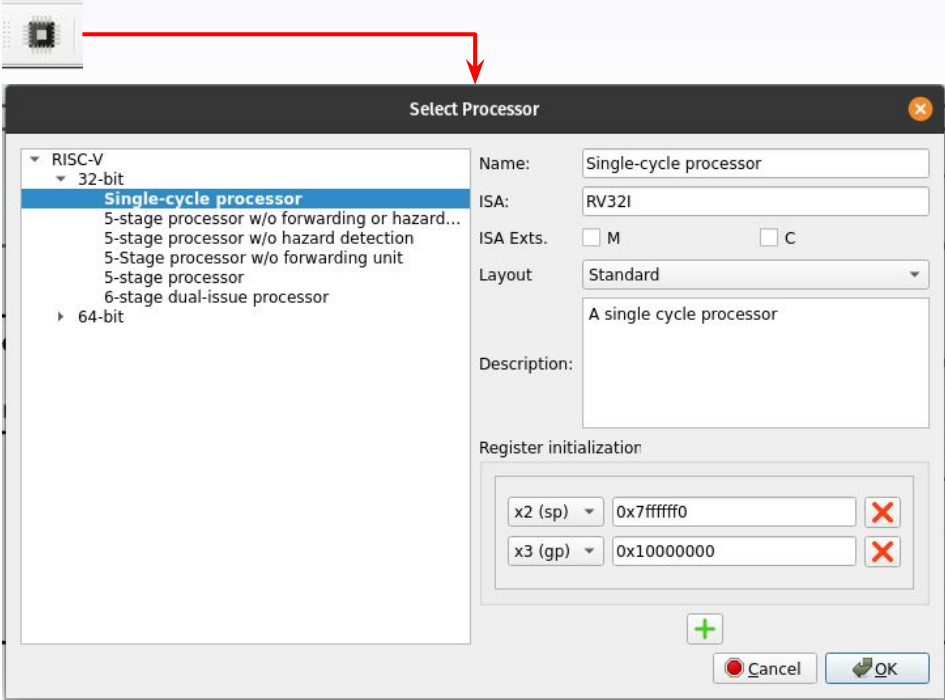
Entorno de trabajo

Ripes y ESP32C3

RIPES: <https://github.com/mortbopet/Ripes>

Ripes es un simulador RISC-V con fines didácticos el cual posee un ensamblador que permite escribir código ASM y traducirlo a código de máquina, permitiendo ejecutar paso a paso las instrucciones. Además puede hacer uso de un toolchain de lenguaje C (GCC) y traducir así C a ASM. Ripes es un proyecto multiplataforma (Linux, Mac, Windows) y de código abierto. Para obtener el ejecutable puede acceder a : <https://github.com/mortbopet/Ripes/releases> (descargar versión 2.2.6 o superior).

Si bien RIPES ofrece simular muchas arquitecturas RISC-V con pipelining y diversas extensiones (C y M), nos limitaremos por ahora a **Single Cycle RISC-V Processor sin extensiones M o C**.



Como todo proyecto de código abierto, se encuentra en continuo desarrollo gracias al aporte de programadores haciéndolo (probablemente) por hobby. Al ser una herramienta principalmente didáctica no pretendemos que represente con fidelidad una microarquitectura real. Por ejemplo, este simulador permite realizar accesos no alineados sin penalidad que en una microarquitectura real no se podrían realizar.

Dado que no existe un sistema operativo, ciertas operaciones de entrada/salida se encuentran simuladas con llamados al sistema.

Ejecución por
pasos

ASM

Código de máquina

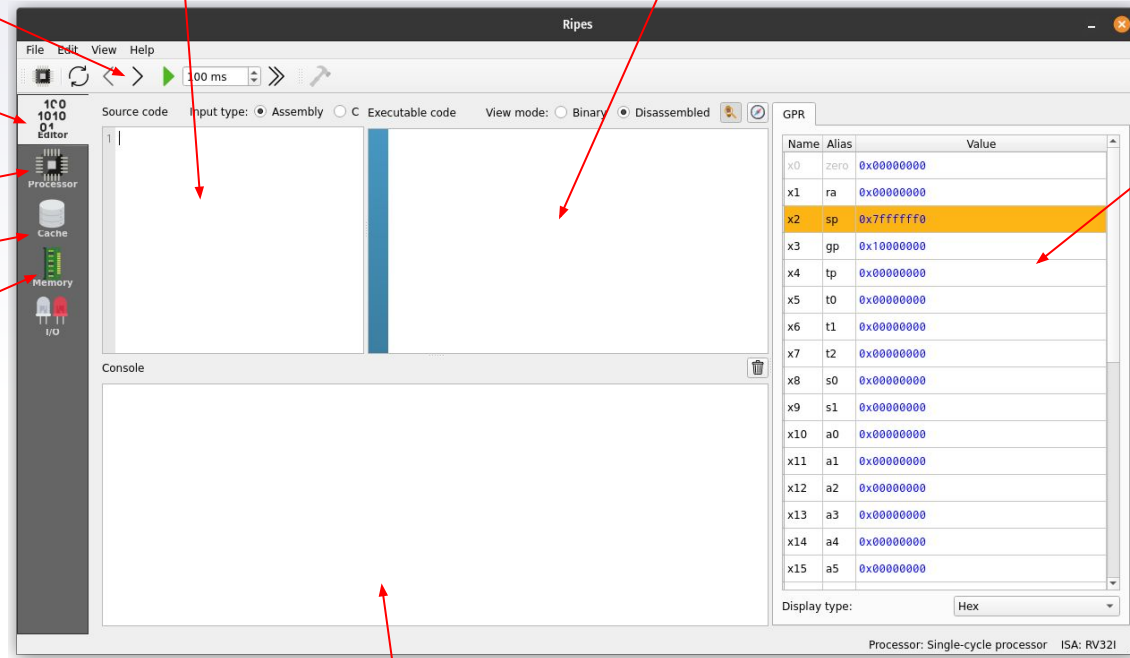
Editor de código

Procesador

Cache

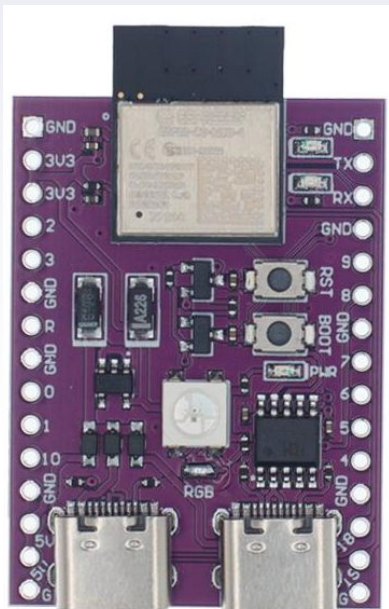
Memoria

Registros



Consola de salida

ESP32C3



Fabricando por Espressif <https://www.espressif.com/en/products/socs/esp32-c3>

Microcontrolador que implementa la arquitectura **RV32imc**.

Usa el chip ESP32-C3-MINI-1-N4 de la familia esp32c3.

- Memoria RAM (RWM) on-chip: 400KB (16KB usados para cache)
- Bootloader (ROM): 384KB
- Memoria Flash : Hasta 16MB (implementados solo 4MB)
- Wifi: IEEE 802.11 b/g/n-compliant
- Bluetooth LE: Bluetooth 5, Bluetooth mesh
- GPIO, SPI, UART, I2C, I2S, remote control peripheral, LED PWM controller, general DMA controller, TWAI® controller (compatible with ISO 11898-1), USB Serial/JTAG controller, temperature sensor, SAR ADC.
- Tensión de alimentación: 3.3V. Corrientes source 40ma, sink 28ma.
- Puerto USB-Serie
- Puerto USB-JTAG (Debugger integrado)
- LED RGB WS2812B
- 11 GPIO disponibles
- Frecuencia Máxima 160MHz (oscilador usado 40MHz)

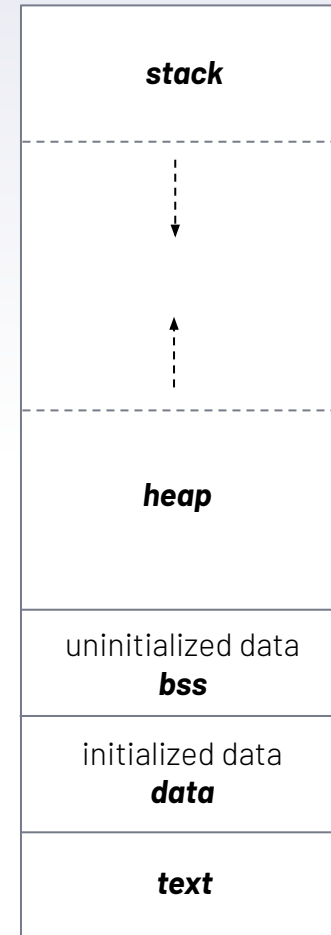


Ripes	ESP32C3
RV32imc	RV32imc
Toda la memoria disponible es RWM	384KB RWM (RAM)
Todo el mapa de memoria implementado	Memoria de datos en 0x3800_0000
	Memoria de instrucciones en 0x4000_0000
Accesos NO alineados soportados SIN penalidad	Accesos NO alineados soportados CON penalidad
Simulador/Debugger paso a paso	Debugger integrado (8 breakpoints)
E/S Simuladas con ECALLs	Biblioteca ESP-IDF
No soporta interrupciones	Soporta interrupciones(32 vectorizadas con 7 prioridades)

Organización de la memoria de un programa

Un programa está compuesto por distintos elementos, los cuales se almacenan en secciones distintas en la memoria

- ▶ **text**: sección que almacena las **instrucciones**.
 - ▷ ej en C: `a = Max(4,i);`
- ▶ **data**: sección que almacena **datos inicializados**
 - ▷ ej en C: `unsigned char vector[ ] = {'A',65,0x41};`
- ▶ **bss**: sección que almacena datos no inicializados
 - ▷ ej en C: `int a;` (en Ripes es idéntica a data)
- ▶ **stack**: espacio para almacenar datos temporales y recuperarlos en orden
 - ▷ ej en C: si llamamos `funcion(1,2,3);` los argumentos se pasan por el stack.
- ▶ **heap**: espacio utilizable de memoria dinámica (ej: `malloc`)





Registros y Valores inmediatos

Ejemplo: Escriba una instrucción que cargue el valor decimal **128 (0x80)** en el registro temporal 0 (**t0 o x5**)

```
1 .text
2  addi t0,x0,128
```

0:

08000293

addi x5 x0 128

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x00000080

Recordemos que el registro x0 (zero) siempre vale 0. Luego sumar $x0+128=128$, el resultado se deposita en el registro x5 (t0). **El formato de ADDI es tipo I.** Este formato soporta inmediatos de **12 bits signados, rango [-2048 : 2047]**

imm[11:0]	rs1	000	rd	0010011	ADDI
imm[11:0]	rs1	funct3	rd	opcode	I-type

Ejercicio: Intente cargar los siguientes valores decimales utilizando addi en el registro x5: 0, 1, -2047, -2048, -2049, 2047, 2048. **Indique** en qué casos es imposible y **explique** cuál es el problema.
Explique qué es lo que ocurre con los 20 bits más significativos del valor inmediato cuando el número es positivo y cuando es negativo.

Ejemplo: Escriba una instrucción que cargue el valor hexadecimal 0x12345000 en el registro x5

```
1 .text
2     lui x5,0x12345
```

0:

123452b7

lui x5 0x12345

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x12345000

LUI (Load Upper Immediate) carga **32 bits** en un registro, pero si la instrucción ocupa 32 bits, entonces el valor inmediato no puede ser también de 32 bits. Por ende solo se pueden utilizar **20 bits (los más significativos)** para el **valor inmediato** y el **resto** se completa con **ceros**.

imm[31:12]	rd	opcode	U-type
imm[31:12]	rd	0110111	LUI

Ejercicio: Combinando LUI y ADDI asigne el valor **0x87654321** en el registro **x5**. Luego utilice la pseudo-instrucción **LI**(load immediate) para realizar la misma operación y compare el ASM generado. **Utilice LI** para asignar los siguientes valores: 2049, 2048, 2047, -2048, -2049, -2047. **Explique** en qué casos puede resolverse utilizando solo ADDI.



Operaciones aritméticas

Ejemplo: Asigne el valor 2047 a x5 y luego decremente en uno su valor.

```
1 .text
2  li x5,2047
3  addi x5,x5,-1
```

0:	7ff00293	addi x5 x0 2047
4:	fff28293	addi x5 x5 -1

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x000007fe

Las instrucciones con operandos inmediatos (ADDI, ANDI, ORI, XORI, SLTI, SRLI, SRAI) asumen que el mismo es signado, por ende restar equivale a sumar un número negativo. Existe el caso de SLTIU donde asume que ambos son números sin signo.

imm[11:0]	rs1	funct3	rd	opcode	I-type
-----------	-----	--------	----	--------	--------

Ejercicio: asigne el valor 10 al registro x5. Luego desplace el mismo a la izquierda (SLLI) en un bit y guarde el resultado en x6. **Desplace** nuevamente x5 pero 2 bits y almacene el resultado en x7. **Repita** este procedimiento 3 veces más pero guardando el resultado en x8,x9, y x10 respectivamente. **Analice** el número que resulta en cada registro. **Almacene** el número 0x8000000F en x5. **Aplique** desplazamiento a la derecha aritmético (SRAI) en un bit sobre x5. **Indique** lo que ocurre con el bit más significativo. Luego **realice** un desplazamiento a la derecha lógico (SRLI) en un bit sobre x5. **Indique** lo que ocurre con el bit más significativo.

Ejemplo: Dado el número positivo 4 almacenado en x5 convierta el mismo a negativo y guárdelo en x4.

```
1 .text
2   li x5,4
3   sub x4,x0,x5
```

0:	00400293	addi x5 x0 4
4:	40500233	sub x4 x0 x5

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0xffffffffc
x5	t0	0x00000004

El formato de instrucción tipo R permite operaciones aritmético-lógicas entre registros (sin inmediatos). El formato es siempre rd = rs1 OP rs2. Las operaciones pueden ser ADD, SUB, SLT(U), SLL, SRA, SRL, AND, OR y XOR.

0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB

Ejercicio: asigne el valor 4 a x5. Luego **utilice** la pseudo-instrucción **NEG** sobre x5. **Analice** la instrucción utilizada por el ensamblador.**Escriba** la tabla de verdad de la función XOR. **Indique** lo que ocurre con la salida cuando el valor de una entrada es 1. **Utilice XORI** para encontrar el complemento a la base menos uno de un número almacenado en x5. Luego **aplique** la pseudo-instrucción **NOT** sobre el mismo registro. **Indique** qué instrucción fue utilizada por el ensamblador.



Accesos a memoria de datos

Ejemplo: Defina una variable de 32 bits (var1) inicializada en 4. Incremente su valor en 1.

```
1 .data
2     var1: .word 4
3
4 .text
5     la x5,var1 #x5 apunta a var1
6     lw x6, 0(x5) #indirecto x5 offset 0
7     addi x6,x6,1
8     sw x6, 0(x5)
```

0:	10000297	auipc x5 0x10000
4:	00028293	addi x5 x5 0
8:	0002a303	lw x6 0 x5
c:	00130313	addi x6 x6 1
10:	0062a023	sw x6 0 x5

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x10000000
x6	t1	0x00000005

La pseudo-instrucción LA almacena en x5 la **dirección** de memoria donde se encuentra almacenada la variable var1.

imm[11:0]	rs1	010	rd	0000011	LW
-----------	-----	-----	----	---------	----

Luego LW x6,0(x5) arma una dirección sumando 0 al contenido de x5 (sin modificar x5) y lee la memoria en esa posición, almacenando su contenido en x6.

Memory map		
Name	Size	Range
.text	20	0x00000000 - 0x00000013
.data	4	0x10000000 - 0x10000003
.bss	0	0x11000000 - 0x10ffffff

En el mapa de memoria vemos que .text se encuentra a partir de 0x00000000. Vemos que .data se encuentra en 0x10000000. Por ende auipc x5 0x10000 genera en x5 el valor el pc (0x00000000) más 0x1000000 y fabrica la dirección 0x10000000 (almacenada en x5) que apunta al comienzo de la variable var1.**PERO** esto lo resuelve el ensamblador!!!. Con usar **LA** nos alcanza.

Ejemplo: Defina una variable de 32 bits (var1) inicializada en 4. Incremente su valor en 1.

```
1 .data
2   var1: .word 4
3
4 .text
5   la x5,var1 #x5 apunta a var1
6   lw x6, 0(x5) #indirecto x5 offset 0
7   addi x6,x6,1
8   sw x6, 0(x5)|
```

0:	10000297	auipc x5 0x10000
4:	00028293	addi x5 x5 0
8:	0002a303	lw x6 0 x5
c:	00130313	addi x6 x6 1
10:	0062a023	sw x6 0 x5

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x10000000
x6	t1	0x00000005

Console

La instrucción SW (Store Word) almacena el contenido de rs2 en la dirección formada por *(rs1+offset)

imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
-----------	-----	-----	-----	----------	---------	----

100
1010
01
Editor

Processor

Cache

Memory

I/O

Memory viewer

Address	Word	Byte 0	Byte 1	Byte 2	Byte 3
0x1000001c	X	X	X	X	X
0x10000018	X	X	X	X	X
0x10000014	X	X	X	X	X
0x10000010	X	X	X	X	X
0x1000000c	X	X	X	X	X
0x10000008	X	X	X	X	X
0x10000004	X	X	X	X	X
0x10000000	0x00000005	0x05	0x00	0x00	0x00

Memory map

Name	Size	Range
.text	20	0x00000000 - 0x00000013
.data	4	0x10000000 - 0x10000003
.bss	0	0x11000000 - 0x10ffffff

Ejemplo: Defina una variable de 32 bits (var1) inicializada en 4. Incremente su valor en 1.

```
1 .data
2   var1: .word 4
3
4 .text
5   la x5,var1 #x5 apunta a var1
6   lw x6, 0(x5) #indirecto x5 offset 0
7   addi x6,x6,1
8   sw x6, 0(x5)
```

0:	10000297	auipc x5 0x10000
4:	00028293	addi x5 x5 0
8:	0002a303	lw x6 0 x5
c:	00130313	addi x6 x6 1
10:	0062a023	sw x6 0 x5

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x10000000
x6	t1	0x00000005

Ejercicio: defina una variable var1 de 32 bits con el contenido 0x66778899. Utilizando **LA** almacene la dirección de var1 en x5. Luego utilizando la instrucción **LB** (Load Byte) **cargue** el contenido de cada byte que compone var1 en los registros consecutivos x6,x7,x8 y x9. **Indique** que ocurre con el valor almacenado en x8 y x9. **Repita** el ejercicio utilizando la instrucción LBU y **analice** el resultado.

Ejercicio 2:Acceda a la variable var1 utilizando la pseudo-instrucción: **lw x5,var1**. **Explique** en qué casos **lw** se toma como instrucción y en que casos como pseudo-instrucción. Luego **utilice sw** en su formato pseudo-instrucción RegDato, Dir, RegAux (sw x6,var1,x5).

Ejemplo: Defina un vector y una variable índice. Acceda al elemento apuntado por el índice y almacene en x4.

```
1 .data
2     indice: .word 3
3     vector: .word 32,64,512,1024,2048
4
5 .text
6     la x5,vector
7     la x6,indice
8     lw x6,0(x6)
9     slli x6,x6,2
10    add x7,x6,x5
11    lw x4,0(x7)
```

0:	10000297	auipc x5 0x10000
4:	00428293	addi x5 x5 4
8:	10000317	auipc x6 0x10000
c:	ff830313	addi x6 x6 -8
10:	00032303	lw x6 0 x6
14:	00231313	slli x6 x6 2
18:	005303b3	add x7 x6 x5
1c:	0003a203	lw x4 0 x7

Name	Alias	Val
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000400
x5	t0	0x10000004
x6	t1	0x0000000c
x7	t2	0x10000010

Console

La instrucción en la línea 8, lw x6,0(x6) va a leer el contenido almacenado en la variable índice y almacenarlo en x6. Si bien queremos acceder al elemento 3 del vector (vector[3]), recuerden que cada dato es de 32 bits, por ende utiliza 4 bytes, y en direccionamiento al byte esto involucra 4 direcciones. Por ende hacemos Shift Left Lógico Inmediato con 2, lo cual desplaza el número a la izquierda dos veces efectivamente multiplicando por 4.

Ejercicio: Cambie el valor del índice por 1 y ejecute el programa. Indique si cambia en algo el código generado. Analice los resultados.



ECall

Ejemplo: Imprima en consola un texto y un valor numérico

```
1 .data
2     texto: .string "Hola Mundo!\n"
3     numero: .word -10
4 .text
5     la a0,texto
6     li a7,4
7     ecall
8
9     la a0,numero
10    lw a0,0(a0)
11    li a7,1
12    ecall
13
```

0:	10000517	auipc x10 0x10000
4:	00050513	addi x10 x10 0
8:	00400893	addi x17 x0 4
c:	00000073	ecall
10:	10000517	auipc x10 0x10000
14:	ffd50513	addi x10 x10 -3
18:	00052503	lw x10 0 x10
1c:	00100893	addi x17 x0 1
20:	00000073	ecall

Console

Hola Mundo!
-10

Supported system calls		
Func. (a7)	Name	PrintString
1	PrintInt	Prints a null-terminated string
2	PrintFloat	Arguments:
4	PrintString	Arg. # Reg. address of the string
10	Exit	
11	PrintChar	
17	GetCWD	Returns:
30	Time_msec	Ret. # Reg.
31	Cycles	
34	PrintIntHex	
35	PrintIntBinary	
36	PrintIntUnsigned	

Dado que RIPES es un simulador, simula una salida de consola utilizando llamadas ECALL. La forma de informarle a RIPES cual es la operación (PrintInt, PrintFloat, PrintString, Exit, etc) es forzar un número en **a7** (1,2,4,10, etc) y pasar el argumento utilizando generalmente **a0**.

- PrintString : en **a0** se pasa la dirección de comienzo de un string (debe terminar con 0). **a7=4**.
- PrintInt : en **a0** se pasa el valor de un entero signado. **a7=1**.

Ejercicio: almacene en un registro el valor 0x12345678 e imprima el mismo usando PrintIntHex.



Saltos condicionales

Ejemplo: Imprima los números de un vector hasta que se encuentre uno negativo (inclusive)

```
1 .data
2     vector: .word 100,200,300,-10,400
3 .text
4     la x5,vector
5     li a7,1
6 loop:
7     lw a0,0(x5)
8     ecall
9     addi x5,x5,4
10    bge a0,x0,loop
11    nop
```

0:	10000297	auipc x5 0x10000
4:	00028293	addi x5 x5 0
8:	00100893	addi x17 x0 1
0000000c <loop>:		
c:	0002a503	lw x10 0 x5
10:	00000073	ecall
14:	00428293	addi x5 x5 4
18:	fe055ae3	bge x10 x0 -12 <loop>
1c:	00000013	addi x0 x0 0

Console

100200300-10|

En los saltos, el primer operando se resta con el segundo operando. Si se cumple la condición en la resta entonces se le suma al PC el offset contenido en la instrucción. En ASM este offset se calcula por el ensamblador (**etiquetas**).

imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU

Ejemplo: Imprima los números de un vector hasta que se encuentre uno negativo (inclusive)

```
1 .data
2     vector: .word 100,200,300,-10,400
3 .text
4     la x5,vector
5     li a7,1
6 loop:
7     lw a0,0(x5)
8     ecall
9     addi x5,x5,4
10    bge a0,x0,loop
11    nop
```

```
0:      10000297      auipc x5 0x10000
4:      00028293      addi x5 x5 0
8:      00100893      addi x17 x0 1

0000000c <loop>:
c:      0002a503      lw x10 0 x5
10:     00000073      ecall
14:     00428293      addi x5 x5 4
18:     fe055ae3      bge x10 x0 -12 <loop>
1c:     00000013      addi x0 x0 0
```

Console

```
100200300-10|
```

Ejercicio: Modifique el programa para que se imprima en pantalla los elementos del vector en orden inverso. **DEBE** utilizar un **ciclo iterativo** (no vale imprimir los valores de forma fija). A tal fin puede:

1. Utilizar una variable índice de vector inicializada en 4 e ir decrementando e imprimiendo hasta que llegue a cero.
2. Inicializar el registro x6 con la dirección de fin del vector e ir decrementando de a 4 x6 hasta que sea igual a x5.
3. Utilizar una variable índice de vector inicializada en 0, almacenar el 5 en un registro e ir incrementando el índice hasta que sea igual al 5 (BEQ) almacenado en registro.



Subrutinas (funciones)

Ejemplo: Escriba una subrutina llamada MIN que reciba números en **a0** y **a1** y devuelva en **a0** el menor.

1	.text	0:	06400513	addi x10 x0 100
2	li a0,100	4:	03200593	addi x11 x0 50
3	li a1,50	8:	00c000ef	jal x1 12 <MIN>
4	jal x1,MIN	c:	00000013	addi x0 x0 0
5	nop			
6	fin:	00000010 <fin>:		
7	beq x0,x0,fin	10:	00000063	beq x0 x0 0 <fin>
8				
9	MIN:	00000014 <MIN>:		
10	blt a0,a1,volver	14:	00b54463	blt x10 x11 8 <volver>
11	addi a0,a1,0	18:	00058513	addi x10 x11 0
12	volver:			
13	jalr x0,x1,0 #ret	0000001c <volver>:		
		1c:	00008067	jalr x0 x1 0

JAL toma el valor del PC+4 y lo almacena en el registro de referencia. Luego suma el offset al PC haciendo que el mismo “salte” a esa posición de memoria. La dirección de retorno queda en el registro (generalmente x1=ra). Para volver utilizamos JALR, que toma el valor almacenado en un registro (x1) y lo suma al offset y aplica ese valor al PC. Permite también almacenar la dirección actual del PC aunque en los retornos esto suele descartarse (usando x0). Dado que es tan común existe la pseudo-instrucción ret.

Ejercicio: Modifique los valores almacenados en a0 y a1 para verificar que la función MIN devuelve el valor correcto. **Pruebe** con valores positivos y negativos. Luego **modifique** la función para que asuma que los argumentos que recibe en a0 y a1 son sin signo.

Ejercicio 2: Escriba una subrutina llamada Mult4 que reciba en a0 un número y devuelva (en a0) un 1 si el número es múltiplo de 4. Recomendamos escribir primero varios números múltiplos de 4 y analizar sus bits. No usar DIV o REM.

Ejercicio 3: Utilice jal MIN (pseudo-instrucción) e **identifique** que registro almacena el retorno.



Calling Convention C

Al incluir subrutinas creadas por terceros, se necesita un **"contrato", "acuerdo" o convención** para definir cómo se le pasan argumentos y como se reciben los resultados. Esto se conoce como **Calling Convention**. En el caso de RISC-V la arquitectura define que los registros **a0~a7** se usan **para pasar argumentos** a la subrutina (en caso de necesitar más se usa el stack). Luego la subrutina **retorna** el resultado utilizando **a0**, pudiendo utilizar también **a1** si el mismo requiere dos registros.

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-7	t0-2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

Considerando que una subrutina va a modificar los registros para lograr su resultado, es necesario definir qué ocurre con los mismos cuando se produce un llamado a subrutina. RISC-V define que **quien llama** a la subrutina (**Caller**) tiene que almacenar previamente los contenidos de **ra, t0~t6 y a0~a7**. Los registros **sp y s0~s11** son respaldados por la subrutina llamada (**Callee**). Los registros restantes, gp y tp, suelen utilizarse por los compiladores para referencias a memoria por ende no suelen ser modificados por las subrutinas.

0.3 Lenguaje de alto nivel

C

Versión 1.0.2

► Comparativa

Lenguaje C

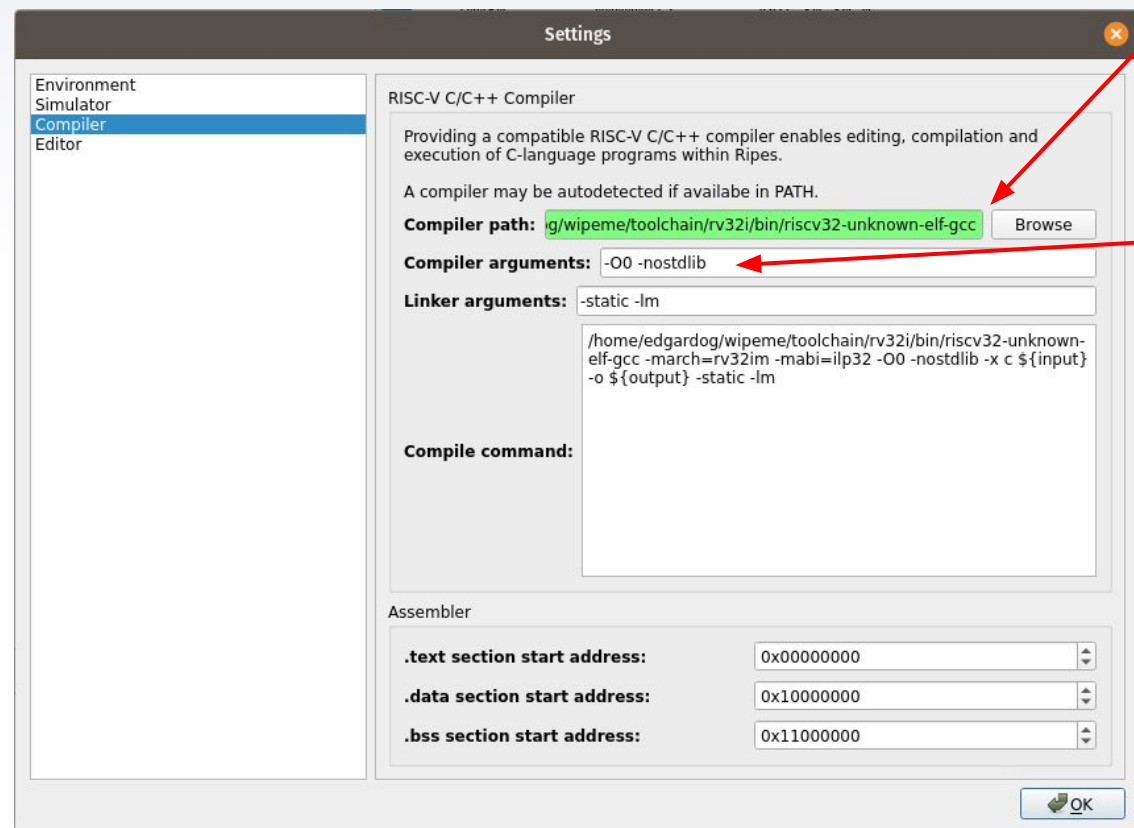
- ▶ Variables locales
- ▶ Operaciones aritméticas o lógicas compuestas
- ▶ Control de flujo (if, while, for, case)
- ▶ Funciones
- ▶ Drivers para acceder al hardware
- ▶ LibC (MSVCRT.dll , libc.so , libSystem.dylib)

ASM

- ▶ Memoria de Datos / Registros
- ▶ Instrucciones aritméticas o lógicas simples
- ▶ Saltos condicionales a etiquetas
- ▶ Subrutinas
- ▶ Hoja de datos

Toolchain GCC RISC-V: <http://tiny.cc/acunlam>

El toolchain es un conjunto de programas que cumplen distintas tareas necesarias para pasar de código de alto nivel (escrito en C) a código de máquina. El compilador GCC (GNU C Compiler) interpreta un programa en C dentro de Ripes y su salida se convierte a ASM para ejecutar dentro del simulador.



Seleccionar el path al binario del compilador (verificar que este en verde)

**Incluir el argumento -nostdlib
Utilizar -O0 (sin optimizar)**

El argumento de gcc **-nostdlib** le indica al compilador que NO utilice llamadas a la biblioteca estándar de C, es decir, la biblioteca que contiene las funciones definidas en stdio.h, stdlib.h, etc. Esto quiere decir que NO tenemos disponible printf, scanf, sprintf, math.h ni nada de todo eso. El argumento **-O0** indica que no se deben realizar optimizaciones. Los compiladores no traducen el código a ASM de forma directa sino que tratan de optimizarlo.

Código en C

Compilar

ASM generado

100
1010
01
Editor

Processor

Cache

Memory

I/O

Source code Input type: ☐ Assembly ☒ C Executable code View mode: ☐ Binary ☒ Disassembled

```
1 void main(){  
2  
3 }
```

00010074 <main>:
10074: ff010113 addi x2 x2 -16
10078: 00812623 sw x8 12 x2
1007c: 01010413 addi x8 x2 16
10080: 00000013 addi x0 x0 0
10084: 00c12403 lw x8 12 x2
10088: 01010113 addi x2 x2 16
1008c: 00008067 jalr x0 x1 0

Console

► Stack Pointer y Frame Pointer

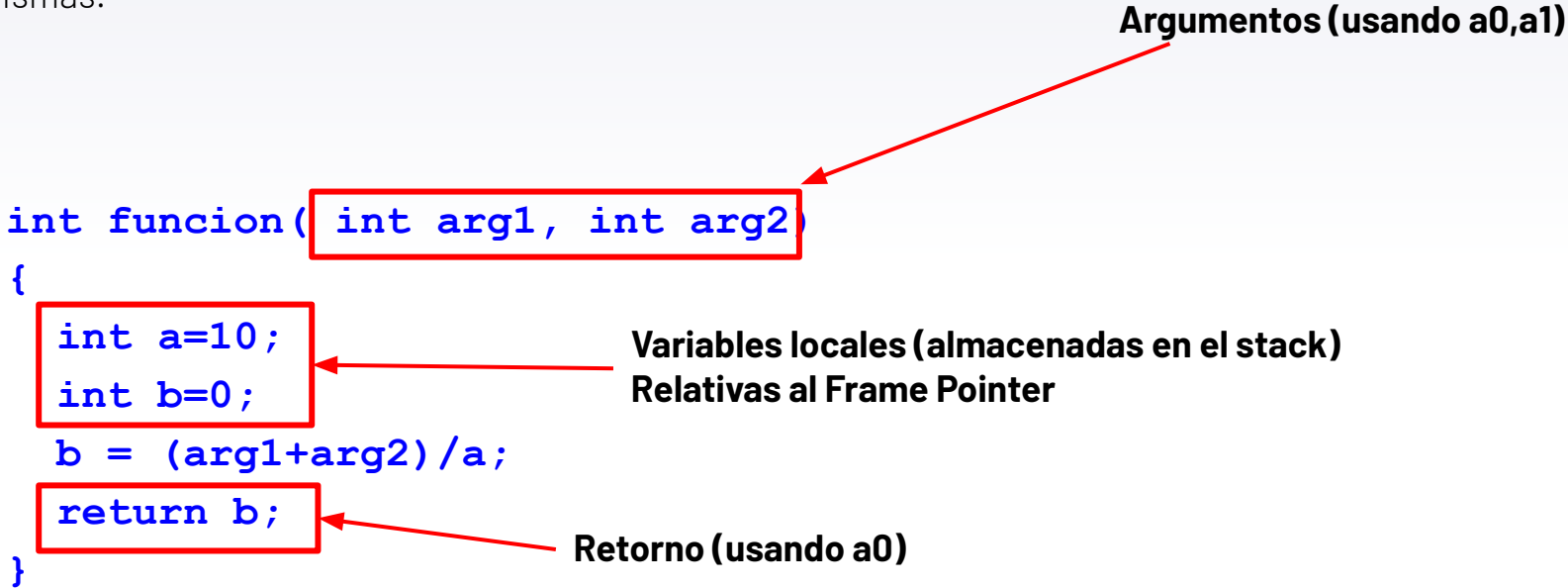
Variables locales

Un programa escrito en lenguaje de alto nivel necesita poder ubicarse en cualquier lugar de la memoria. Esto quiere decir que si utiliza variables para almacenar información, estas variables NO deben estar en posiciones absolutas de memoria sino en direcciones relativas a algún registro. De esa forma cuando el loader (sistema operativo) carga el programa en memoria para ejecutarse, las direcciones absolutas serán distintas pero las relativas serán siempre las mismas.

```
int funcion( int arg1, int arg2)
{
    int a=10;
    int b=0;
    b = (arg1+arg2)/a;
    return b;
}
```

Variables locales

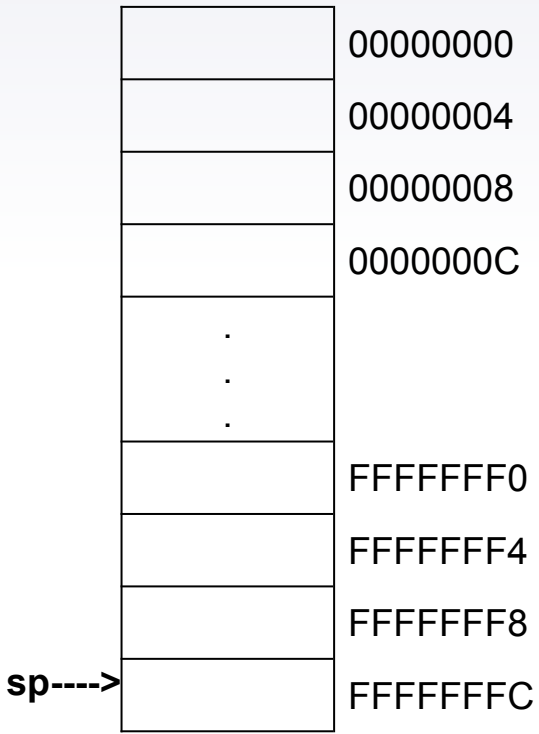
Un programa escrito en lenguaje de alto nivel necesita poder ubicarse en cualquier lugar de la memoria. Esto quiere decir que si utiliza variables para almacenar información, estas variables NO deben estar en posiciones absolutas de memoria sino en direcciones relativas a algún registro. De esa forma cuando el loader (sistema operativo) carga el programa en memoria para ejecutarse, las direcciones absolutas serán distintas pero las relativas serán siempre las mismas.



Variables locales

El **stack pointer (sp)** se ubica en la dirección más alta de la memoria asignada para la función. Cuando comienza la función se reservan en el stack pointer el espacio para las variables locales...

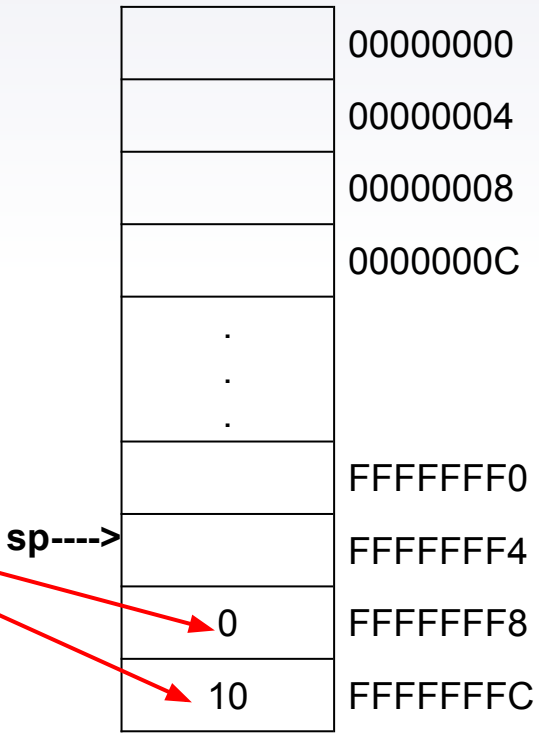
```
int funcion( int arg1, int arg2)
{
    int a=10;
    int b=0;
    b = (arg1+arg2)/a;
    return b;
}
```



Variables locales

El **stack pointer (sp)** se decrementa reservando el espacio para estas variables y almacena el valor de las mismas. Por ende el acceso a las variables locales ahora se puede hacer relativo al sp. Si usamos **sp[4]** accedemos a la variable **b**, y con **sp[8]** accedemos a la variable **a**.

```
int funcion( int arg1, int arg2)
{
    int a=10;    a = sp [8]
    int b=0;     b = sp [4]
    b = (arg1+arg2)/a;
    return b;
}
```

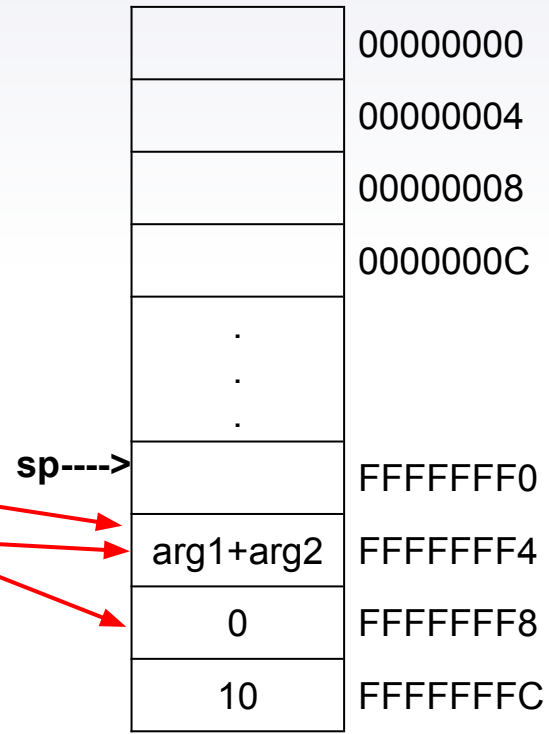


Variables locales

El **stack pointer (sp)** se decrementa reservando el espacio para estas variables y almacena el valor de las mismas. Por ende el acceso a las variables locales ahora se puede hacer relativo al sp. Si usamos **sp[4]** accedemos a la variable **b**, y con **sp[8]** accedemos a la variable **a**. **Pero si el stack pointer cambia.... Ahora esas referencias a las variables ya no se encuentran en sp[4] y sp[8].**

```
int funcion( int arg1, int arg2)
{
    int a=10;
    int b=0;
    b = (arg1+arg2)/a;
    return b;
}
```

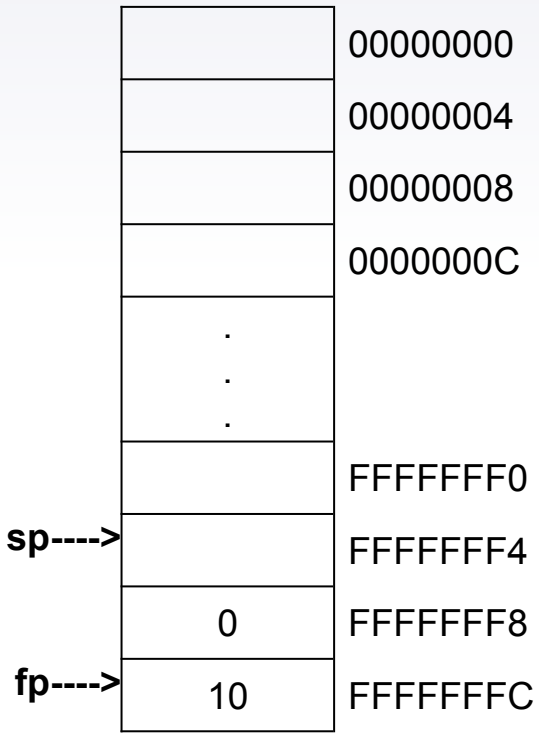
a = sp [8]
b = sp [4]



Variables locales

Para evitar este problema se utiliza el **frame pointer(fp)**. El mismo se ubica en el mismo lugar donde se ubica el stack al momento de comenzar la función. Entonces ahora el stack pointer puede cambiar, ya que nos referimos a las variables utilizando el frame pointer.

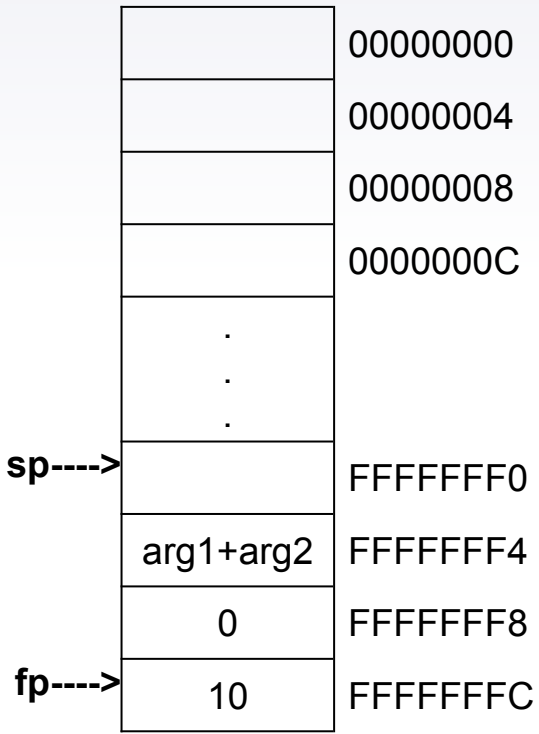
```
int funcion( int arg1, int arg2)
{
    int a=10;  a = fp [0]
    int b=0;   b = fp [-4]
    b = (arg1+arg2)/a;
    return b;
}
```



Variables locales

Para evitar este problema se utiliza el **frame pointer(fp)**. El mismo se ubica en el mismo lugar donde se ubica el stack al momento de comenzar la función. Entonces ahora el stack pointer puede cambiar, ya que nos referimos a las variables utilizando el frame pointer.

```
int funcion( int arg1, int arg2)
{
    int a=10;  a = fp [0]
    int b=0;   b = fp [-4]
    b = (arg1+arg2)/a;
    return b;
}
```



► C → ASM RISC-V

```
void main(){
```

```
}|
```

00010074 <main>:

10074:	ff010113	addi x2 x2 -16
10078:	00812623	sw x8 12 x2
1007c:	01010413	addi x8 x2 16
10080:	00000013	addi x0 x0 0
10084:	00c12403	lw x8 12 x2
10088:	01010113	addi x2 x2 16
1008c:	00008067	jalr x0 x1 0

```
void main(){
```

```
}|
```

```
00010074 <main>:
```

10074:	ff010113	addi x2 x2 -16
10078:	00812623	sw x8 12 x2
1007c:	01010413	addi x8 x2 16
10080:	00000013	addi x0 x0 0
10084:	00c12403	lw x8 12 x2
10088:	01010113	addi x2 x2 16
1008c:	00008067	jalr x0 x1 0

Empezando por el final, **jalr x0,x1, 0** retorna a quien sea que haya llamado a main().

Siempre que termina una función en C hay un **return**, aun cuando no sea explícito a algún lado tiene que retornar. Se asume según el calling convention que el registro **ra** (x1) almacena la dirección de retorno.

```
void main(){  
}
```

00010074 <main>:

10074:	ff010113	addi x2 x2 -16
10078:	00812623	sw x8 12 x2
1007c:	01010413	addi x8 x2 16
10080:	00000013	addi x0 x0 0
10084:	00c12403	lw x8 12 x2
10088:	01010113	addi x2 x2 16
1008c:	00008067	jalc x0 x1 0

Vemos que cuando comienza la función, se decrementa el **stack pointer (x2)** en **16**, ya que se reserva el espacio de trabajo para variables locales.

Vemos que cuando está por terminar la función le vuelve a sumar 16 al stack pointer para restaurar el valor original que tenía al comenzar la función.

Vemos que el calling convention especifica que el valor del sp (x2) se debe preservar por la función.

x2 | sp | Stack pointer | Callee

```
void main(){  
}
```

00010074 <main>:

10074:	ff010113	addi x2 x2 -16
10078:	00812623	sw x8 12 x2
1007c:	01010413	addi x8 x2 16
10080:	00000013	addi x0 x0 0
10084:	00c12403	lw x8 12 x2
10088:	01010113	addi x2 x2 16
1008c:	00008067	jlr x0 x1 0

En el tope del stack (offset 12) almacena el contenido del frame pointer (fp x8) al momento de entrar en la función.

Restaura el valor original del frame pointer antes de retornar

x8	s0/fp	Saved register/frame pointer	Callee
----	-------	------------------------------	--------

```
void main(){  
}
```

00010074 <main>:

10074:	ff010113	addi x2 x2 -16
10078:	00812623	sw x8 12 x2
1007c:	01010413	addi x8 x2 16
10080:	00000013	addi x0 x0 0
10084:	00c12403	lw x8 12 x2
10088:	01010113	addi x2 x2 16
1008c:	00008067	jalr x0 x1 0



El frame pointer ahora toma la dirección original del stack pointer. Ahora el Frame pointer está apuntando al comienzo original del stack, y el stack puede crecer libremente pero usamos el frame pointer para las referencias a las variables locales. De hecho lo primero que apunta ahora el FP es el valor que tenía originalmente al ingresar a la función.

```
void main(){
```

```
}|
```

```
00010074 <main>:
```

10074:	ff010113	addi x2 x2 -16
10078:	00812623	sw x8 12 x2
1007c:	01010413	addi x8 x2 16
10080:	00000013	addi x0 x0 0
10084:	00c12403	lw x8 12 x2
10088:	01010113	addi x2 x2 16
1008c:	00008067	jalr x0 x1 0

NOP

Dado que la función no hace nada, ejecuta un NOP y comienza el proceso de restaurar el stack pointer y frame pointer a sus valores originales para cumplir con el calling convention.

► Variables locales y aritmética

```

1 int main(){
2     int a=8;
3     int b=10;
4
5     int c = a+b;
6
7     return c;|
8 }

```

```

00010074 <main>:
10074:      fe010113      addi x2, x2, -32
10078:      00812e23      sw x8, 28(x2)
1007c:      02010413      addi x8, x2, 32
10080:      00800793      addi x15, x0, 8
10084:      fef42623      sw x15, -20(x8)
10088:      00a00793      addi x15, x0, 10
1008c:      fef42423      sw x15, -24(x8)
10090:      fec42703      lw x14, -20(x8)
10094:      fe842783      lw x15, -24(x8)
10098:      00f707b3      add x15, x14, x15
1009c:      fef42223      sw x15, -28(x8)
100a0:      fe442783      lw x15, -28(x8)
100a4:      00078513      addi x10, x15, 0
100a8:      01c12403      lw x8, 28(x2)
100ac:      02010113      addi x2, x2, 32
100b0:      00008067      jalr x0, x1, 0

```

El return utilizando **ra**.

```

1 int main(){
2     int a=8;
3     int b=10;
4
5     int c = a+b;
6
7     return c;|
8 }

```

```

00010074 <main>:
10074:      fe010113      addi x2, x2, -32
10078:      00812e23      sw x8, 28(x2)
1007c:      02010413      addi x8, x2, 32
10080:      00800793      addi x15, x0, 8
10084:      fef42623      sw x15, -20(x8)
10088:      00a00793      addi x15, x0, 10
1008c:      fef42423      sw x15, -24(x8)
10090:      fec42703      lw x14, -20(x8)
10094:      fe842783      lw x15, -24(x8)
10098:      00f707b3      add x15, x14, x15
1009c:      fef42223      sw x15, -28(x8)
100a0:      fe442783      lw x15, -28(x8)
100a4:      00078513      addi x10, x15, 0
100a8:      01c12403      lw x8, 28(x2)
100ac:      02010113      addi x2, x2, 32
100b0:      00008067

```

Se prepara el stack pointer y el frame pointer. Recordemos que el frame pointer (**fp x8**) es la base donde van a comenzar las variables locales.

```

1 int main(){
2     int a=8;
3     int b=10;
4
5     int c = a+b;
6
7     return c;
8 }

```

a = fp [-20]

00010074 <main>:

10074:	fe010113	addi x15, x0, 8
10078:	00812e23	sw x15, -20(x8)
1007c:	02010413	addi x15, x0, 10
10080:	00800793	sw x15, -24(x8)
10084:	fef42623	lw x14, -20(x8)
10088:	00a00793	lw x15, -24(x8)
1008c:	fef42423	add x15, x14, x15
10090:	fec42703	sw x15, -28(x8)
10094:	fe842783	lw x15, -28(x8)
10098:	00f707b3	addi x10, x15, 0
1009c:	fef42223	
100a0:	fe442783	
100a4:	00078513	
100a8:	01c12403	
100ac:	02010113	
100b0:	00008067	

Vemos que el **a=8** genera un **8** en **x15**, y luego ese valor se deposita en **x8 (fp) menos 20**. Así que la variable **a** se encuentra en memoria en la dirección apuntada por el frame pointer (que puede cambiar en distintas ejecuciones) menos 20.

```

1 int main(){
2     int a=8;
3     int b=10;
4
5     int c = a+b;
6
7     return c;|
8 }

```

a = fp [-20]
b = fp [-24]

```

00010074 <main>:
10074:    fe010113
10078:    00812e23
1007c:    02010413
10080:    00800793
10084:    fef42623
10088:    00a00793
1008c:    fef42423
10090:    fec42703
10094:    fe842783
10098:    00f707b3
1009c:    fef42223
100a0:    fe442783
100a4:    00078513
100a8:    01c12403
100ac:    02010113
100b0:    00008067

```

addi x15, x0, 10
 sw x15, -24, x8

Vemos que el b=10 es similar... Se almacena en fp - 24.

```

1 int main(){
2     int a=8;
3     int b=10;
4
5     int c = a+b;
6
7     return c;|
8 }

```

a = fp [-20]
b = fp [-24]
c = fp [-28]



```

00010074 <main>:
10074:    fe010113
10078:    00812e23
1007c:    02010413
10080:    00800793
10084:    fef42623
10088:    00a00793
1008c:    fef42423
10090:    fec42703
10094:    fe842783
10098:    00f707b3
1009c:    fef42223
100a0:    fe442783
100a4:    00078513
100a8:    01c12403
100ac:    02010113
100b0:    00008067

```

lw x14 -20 x8
lw x15 -24 x8
add x15 x14 x15
sw x15 -28 x8

Con dos lw se recuperan los valores de **a** y **b** (respectivamente a x14 y x15) y se suman guardando el resultado en x15. Luego el valor de x15 se deposita en fp -28 (lugar donde almacena la variable **c**).

```

1 int main(){
2     int a=8;      a = fp [-20]
3     int b=10;     b = fp [-24]
4
5     int c = a+b;   c = fp [-28]
6
7     return c;|
8 }

```

```

00010074 <main>:
10074:      fe010113      
10078:      00812e23      
1007c:      02010413      
10080:      00800793      
10084:      fef42623      
10088:      00a00793      
1008c:      fef42423      
10090:      fec42703      
10094:      fe842783      
10098:      00f707b3      
1009c:      fef42223      
100a0:      fe442783      
100a4:      00078513      
100a8:      01c12403      
100ac:      02010113      
100b0:      00008067      

```

Por último recupera en x15 el valor de **c** (totalmente de más ya que el valor estaba ahí, pero este código se genera con la opción -O0 , o sea sin optimizaciones). Luego ese valor se copia de x15 a x10... Vemos que x10 es el registro donde se devuelven los resultados de función según el calling convention.

x10-11 | a0-1

Argumentos de función/valores de retorno

► Ciclo iterativo (while)

```

1 int main(){
2     int a=8;
3     int b=10;
4     int c=0;
5
6     while (a>0){
7         c=c+b;
8         a--;
9     }
10
11     return c;
12 }

```

00010074 <main>:

10074:	fe010113	addi x2 x2 -32
10078:	00812e23	sw x8 28 x2
1007c:	02010413	addi x8 x2 32
10080:	00800793	addi x15 x0 8
10084:	fef42623	sw x15 -20 x8
10088:	00a00793	addi x15 x0 10
1008c:	fef42223	sw x15 -28 x8
10090:	fe042423	sw x0 -24 x8
10094:	0200006f	jal x0 32
10098:	fe842703	lw x14 -24 x8
1009c:	fe442783	lw x15 -28 x8
100a0:	00f707b3	add x15 x14 x15
100a4:	fef42423	sw x15 -24 x8
100a8:	fec42783	lw x15 -20 x8
100ac:	fff78793	addi x15 x15 -1
100b0:	fef42623	sw x15 -20 x8
100b4:	fec42783	lw x15 -20 x8
100b8:	fef040e3	blt x0 x15 -32
100bc:	fe842783	lw x15 -24 x8
100c0:	00078513	addi x10 x15 0
100c4:	01c12403	lw x8 28 x2
100c8:	02010113	addi x2 x2 32
100cc:	00008067	jalr x0 x1 0

a = fp [-20]

1	<code>int main(){</code>	00010074 <main>:			
2	<code>int a=8;</code>	10074:	fe010113	<code>addi x2 x2 -32</code>	a = fp [-20] b = fp [-28]
3	<code>int b=10;</code>	10078:	00812e23	<code>sw x8 28 x2</code>	
4	<code>int c=0;</code>	1007c:	02010413	<code>addi x8 x2 32</code>	
5		10080:	00800793	<code>addi x15 x0 8</code>	
6	<code>while (a>0){</code>	10084:	fef42623	<code>sw x15 -20 x8</code>	
7	<code> c=c+b;</code>	10088:	00a00793	<code>addi x15 x0 10</code>	
8	<code> a--;</code>	1008c:	fef42223	<code>sw x15 -28 x8</code>	
9	<code>}</code>	10090:	fe042423	<code>sw x0 -24 x8</code>	
10		10094:	0200006f	<code>jal x0 32</code>	
11	<code>return c;</code>	10098:	fe842703	<code>lw x14 -24 x8</code>	
12	<code>}</code>	1009c:	fe442783	<code>lw x15 -28 x8</code>	
		100a0:	00f707b3	<code>add x15 x14 x15</code>	
		100a4:	fef42423	<code>sw x15 -24 x8</code>	
		100a8:	fec42783	<code>lw x15 -20 x8</code>	
		100ac:	fff78793	<code>addi x15 x15 -1</code>	
		100b0:	fef42623	<code>sw x15 -20 x8</code>	
		100b4:	fec42783	<code>lw x15 -20 x8</code>	
		100b8:	fef040e3	<code>blt x0 x15 -32</code>	
		100bc:	fe842783	<code>lw x15 -24 x8</code>	
		100c0:	00078513	<code>addi x10 x15 0</code>	
		100c4:	01c12403	<code>lw x8 28 x2</code>	
		100c8:	02010113	<code>addi x2 x2 32</code>	
		100cc:	00008067	<code>jalr x0 x1 0</code>	

1	<code>int main(){</code>	00010074 <main>:			
2	<code>int a=8;</code>	10074:	fe010113	<code>addi x2 x2 -32</code>	a = fp [-20]
3	<code>int b=10;</code>	10078:	00812e23	<code>sw x8 28 x2</code>	b = fp [-28]
4	<code>int c=0;</code>	1007c:	02010413	<code>addi x8 x2 32</code>	
5		10080:	00800793	<code>addi x15 x0 8</code>	c = fp [-24]
6	<code>while (a>0){</code>	10084:	fef42623	<code>sw x15 -20 x8</code>	
7	<code> c=c+b;</code>	10088:	00a00793	<code>addi x15 x0 10</code>	
8	<code> a--;</code>	1008c:	fef42223	<code>sw x15 -28 x8</code>	
9	<code>}</code>	10090:	fe042423	<code>sw x0 -24 x8</code>	
10		10094:	0200006f	<code>jal x0 32</code>	
11	<code>return c;</code>	10098:	fe842703	<code>lw x14 -24 x8</code>	
12	<code>}</code>	1009c:	fe442783	<code>lw x15 -28 x8</code>	
		100a0:	00f707b3	<code>add x15 x14 x15</code>	
		100a4:	fef42423	<code>sw x15 -24 x8</code>	
		100a8:	fec42783	<code>lw x15 -20 x8</code>	
		100ac:	fff78793	<code>addi x15 x15 -1</code>	
		100b0:	fef42623	<code>sw x15 -20 x8</code>	
		100b4:	fec42783	<code>lw x15 -20 x8</code>	
		100b8:	fef040e3	<code>blt x0 x15 -32</code>	
		100bc:	fe842783	<code>lw x15 -24 x8</code>	
		100c0:	00078513	<code>addi x10 x15 0</code>	
		100c4:	01c12403	<code>lw x8 28 x2</code>	
		100c8:	02010113	<code>addi x2 x2 32</code>	
		100cc:	00008067	<code>jalr x0 x1 0</code>	

```

1 int main(){
2     int a=8;
3     int b=10;
4     int c=0;
5
6     while (a>0){
7         c=c+b;
8         a--;
9     }
10
11     return c;
12 }

```

00010074 <main>:

10074:	fe010113	addi x2 x2 -32
10078:	00812e23	sw x8 28 x2
1007c:	02010413	addi x8 x2 32
10080:	00800793	addi x15 x0 8
10084:	fef42623	sw x15 -20 x8
10088:	00a00793	addi x15 x0 10
1008c:	fef42223	sw x15 -28 x8
10090:	fe042423	sw x0 -24 x8
10094:	0200006f	jal x0 32
10098:	fe842703	lw x14 -24 x8
1009c:	fe442783	lw x15 -28 x8
100a0:	00f707b3	add x15 x14 x15
100a4:	fef42423	sw x15 -24 x8
100a8:	fec42783	lw x15 -20 x8
100ac:	fff78793	addi x15 x15 -1
100b0:	fef42623	sw x15 -20 x8
100b4:	fec42783	lw x15 -20 x8
100b8:	fef040e3	blt x0 x15 -32
100bc:	fe842783	lw x15 -24 x8
100c0:	00078513	addi x10 x15 0
100c4:	01c12403	lw x8 28 x2
100c8:	02010113	addi x2 x2 32
100cc:	00008067	jalr x0 x1 0

a = fp [-20]
 b = fp [-28]
 c = fp [-24]

pc = 10094

pc = 10094 + 20 (32_{10})

```

1 int main(){
2     int a=8;
3     int b=10;
4     int c=0;
5
6     while (a>0){
7         c=c+b;
8         a--;
9     }
10
11     return c;
12 }

```

00010074 <main>:

10074:	fe010113	addi x2 x2 -32
10078:	00812e23	sw x8 28 x2
1007c:	02010413	addi x8 x2 32
10080:	00800793	addi x15 x0 8
10084:	fef42623	sw x15 -20 x8
10088:	00a00793	addi x15 x0 10
1008c:	fef42223	sw x15 -28 x8
10090:	fe042423	sw x0 -24 x8
10094:	0200006f	jal x0 32
10098:	fe842703	lw x14 -24 x8
1009c:	fe442783	lw x15 -28 x8
100a0:	00f707b3	add x15 x14 x15
100a4:	fef42423	sw x15 -24 x8
100a8:	fec42783	lw x15 -20 x8
100ac:	fff78793	addi x15 x15 -1
100b0:	fef42623	sw x15 -20 x8
100b4:	fec42783	lw x15 -20 x8
100b8:	fef040e3	blt x0 x15 -32
100bc:	fe842783	lw x15 -24 x8
100c0:	00078513	addi x10 x15 0
100c4:	01c12403	lw x8 28 x2
100c8:	02010113	addi x2 x2 32
100cc:	00008067	jalr x0 x1 0

a = fp [-20]
 b = fp [-28]
 c = fp [-24]

x15 = a
 if (0<a) goto 100b8-20 (10098)
 else goto 100bc

```

1 int main(){
2     int a=8;
3     int b=10;
4     int c=0;
5
6     while (a>0){
7         c=c+b;
8         a--;
9     }
10
11     return c;
12 }

```

00010074 <main>:

10074:	fe010113	addi x2 x2 -32
10078:	00812e23	sw x8 28 x2
1007c:	02010413	addi x8 x2 32
10080:	00800793	addi x15 x0 8
10084:	fef42623	sw x15 -20 x8
10088:	00a00793	addi x15 x0 10
1008c:	fef42223	sw x15 -28 x8
10090:	fe042423	sw x0 -24 x8
10094:	0200006f	jal x0 32
10098:	fe842703	lw x14 -24 x8
1009c:	fe442783	lw x15 -28 x8
100a0:	00f707b3	add x15 x14 x15
100a4:	fef42423	sw x15 -24 x8
100a8:	fec42783	lw x15 -20 x8
100ac:	fff78793	addi x15 x15 -1
100b0:	fef42623	sw x15 -20 x8
100b4:	fec42783	lw x15 -20 x8
100b8:	fef040e3	blt x0 x15 -32
100bc:	fe842783	lw x15 -24 x8
100c0:	00078513	addi x10 x15 0
100c4:	01c12403	lw x8 28 x2
100c8:	02010113	addi x2 x2 32
100cc:	00008067	jalr x0 x1 0

a = fp [-20]
b = fp [-28]
c = fp [-24]

x14 = c
x15 = b
x15 = x14+x15
c = x15

```

1 int main(){
2     int a=8;
3     int b=10;
4     int c=0;
5
6     while (a>0){
7         c=c+b;
8         a--;
9     }
10
11     return c;
12 }

```

00010074 <main>:

10074:	fe010113	addi x2 x2 -32
10078:	00812e23	sw x8 28 x2
1007c:	02010413	addi x8 x2 32
10080:	00800793	addi x15 x0 8
10084:	fef42623	sw x15 -20 x8
10088:	00a00793	addi x15 x0 10
1008c:	fef42223	sw x15 -28 x8
10090:	fe042423	sw x0 -24 x8
10094:	0200006f	jal x0 32
10098:	fe842703	lw x14 -24 x8
1009c:	fe442783	lw x15 -28 x8
100a0:	00f707b3	add x15 x14 x15
100a4:	fef42423	sw x15 -24 x8
100a8:	fec42783	lw x15 -20 x8
100ac:	fff78793	addi x15 x15 -1
100b0:	fef42623	sw x15 -20 x8
100b4:	fec42783	lw x15 -20 x8
100b8:	fef040e3	blt x0 x15 -32
100bc:	fe842783	lw x15 -24 x8
100c0:	00078513	addi x10 x15 0
100c4:	01c12403	lw x8 28 x2
100c8:	02010113	addi x2 x2 32
100cc:	00008067	jalr x0 x1 0

a = fp [-20]
b = fp [-28]
c = fp [-24]

x15 = a
x15 = x15 -1
a = x15

► Llamado a función

```

1 int funcion(int,int);
2
3 int main(){
4     int a=8;
5     int b=10;
6     int c=0;
7
8     while (a>0){
9         c=funcion(c,b);
10        a--;
11    }
12
13    return c;
14 }
15
16 int funcion(int a,int b){
17     return a+b;
18 }
19

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00112e23      sw x1 28 x2
1007c:      00812c23      sw x8 24 x2
10080:      02010413      addi x8 x2 32
10084:      00800793      addi x15 x0 8
10088:      fef42623      sw x15 -20 x8
1008c:      00a00793      addi x15 x0 10
10090:      fef42223      sw x15 -28 x8
10094:      fe042423      sw x0 -24 x8
10098:      0200006f      jal x0 32
1009c:      fe442583      lw x11 -28 x8
100a0:      fe842503      lw x10 -24 x8
100a4:      034000ef      jal x1 52 <funcion>
100a8:      fea42423      sw x10 -24 x8
100ac:      fec42783      lw x15 -20 x8
100b0:      fff78793      addi x15 x15 -1
100b4:      fef42623      sw x15 -20 x8
100b8:      fec42783      lw x15 -20 x8
100bc:      fef040e3      blt x0 x15 -32
100c0:      fe842783      lw x15 -24 x8
100c4:      00078513      addi x10 x15 0
100c8:      01c12083      lw x1 28 x2
100cc:      01812403      lw x8 24 x2
100d0:      02010113      addi x2 x2 32
100d4:      00008067      jalr x0 x1 0

000100d8 <funcion>:
100d8:      fe010113      addi x2 x2 -32
100dc:      00812e23      sw x8 28 x2
100e0:      02010413      addi x8 x2 32
100e4:      fea42623      sw x10 -20 x8
100e8:      feb42423      sw x11 -24 x8
100ec:      fec42703      lw x14 -20 x8
100f0:      fe842783      lw x15 -24 x8
100f4:      00f707b3      add x15 x14 x15
100f8:      00078513      addi x10 x15 0
100fc:      01c12403      lw x8 28 x2
10100:      02010113      addi x2 x2 32
10104:      00008067      jalr x0 x1 0

```

```

1 int funcion(int,int);
2
3 int main(){
4     int a=8;
5     int b=10;
6     int c=0;
7
8     while (a>0){
9         c=funcion(c,b);
10        a--;
11    }
12
13    return c;
14 }
15
16 int funcion(int a,int b){
17     return a+b;
18 }
19

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00112e23      sw x1 28 x2
1007c:      00812c23      sw x8 24 x2
10080:      02010413      addi x8 x2 32
10084:      00800793      addi x15 x0 8
10088:      fef42623      sw x15 -20 x8
1008c:      00a00793      addi x15 x0 10
10090:      fef42223      sw x15 -28 x8
10094:      fe042423      sw x0 -24 x8
10098:      0200006f      jal x0 32
1009c:      fe442583      lw x11 -28 x8
100a0:      fe842503      lw x10 -24 x8
100a4:      034000ef      jal x1 52 <funcion>
100a8:      fea42423      sw x10 -24 x8
100ac:      fec42783      lw x15 -20 x8
100b0:      fff78793      addi x15 x15 -1
100b4:      fef42623      sw x15 -20 x8
100b8:      fec42783      lw x15 -20 x8
100bc:      fef040e3      blt x0 x15 -32
100c0:      fe842783      lw x15 -24 x8
100c4:      00078513      addi x10 x15 0
100c8:      01c12083      lw x1 28 x2
100cc:      01812403      lw x8 24 x2
100d0:      02010113      addi x2 x2 32
100d4:      00008067      jalr x0 x1 0

000100d8 <funcion>:
100d8:      fe010113      addi x2 x2 -32
100dc:      00812e23      sw x8 28 x2
100e0:      02010413      addi x8 x2 32
100e4:      fea42623      sw x10 -20 x8
100e8:      feb42423      sw x11 -24 x8
100ec:      fec42703      lw x14 -20 x8
100f0:      fe842783      lw x15 -24 x8
100f4:      00f707b3      add x15 x14 x15
100f8:      00078513      addi x10 x15 0
100fc:      01c12403      lw x8 28 x2
10100:      02010113      addi x2 x2 32
10104:      00008067      jalr x0 x1 0

```

a = fp [-20]
b = fp [-28]
c = fp [-24]

```

1 int function(int,int);
2
3 int main(){
4     int a=8;
5     int b=10;
6     int c=0;
7
8     while (a>0){
9         c=function(c,b);
10        a--;
11    }
12
13    return c;
14 }
15
16 int function(int a,int b){
17     return a+b;
18 }
19

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00112e23      sw x1 28 x2
1007c:      00812c23      sw x8 24 x2
10080:      02010413      addi x8 x2 32
10084:      00800793      addi x15 x0 8
10088:      fef42623      sw x15 -20 x8
1008c:      00a00793      addi x15 x0 10
10090:      fef42223      sw x15 -28 x8
10094:      fe042423      sw x0 -24 x8
10098:      0200006f      jal x0 32
1009c:      fe442583      lw x11 -28 x8
100a0:      fe842503      lw x10 -24 x8
100a4:      034000ef      jal x1 52 <funcion>
100a8:      fea42423      sw x10 -24 x8
100ac:      fec42783      lw x15 -20 x8
100b0:      fff78793      addi x15 x15 -1
100b4:      fef42623      sw x15 -20 x8
100b8:      fec42783      lw x15 -20 x8
100bc:      fef040e3      blt x0 x15 -32
100c0:      fe842783      lw x15 -24 x8
100c4:      00078513      addi x10 x15 0
100c8:      01c12083      lw x1 28 x2
100cc:      01812403      lw x8 24 x2
100d0:      02010113      addi x2 x2 32
100d4:      00008067      jalr x0 x1 0

000100d8 <funcion>:
100d8:      fe010113      addi x2 x2 -32
100dc:      00812e23      sw x8 28 x2
100e0:      02010413      addi x8 x2 32
100e4:      fea42623      sw x10 -20 x8
100e8:      feb42423      sw x11 -24 x8
100ec:      fec42703      lw x14 -20 x8
100f0:      fe842783      lw x15 -24 x8
100f4:      00f707b3      add x15 x14 x15
100f8:      00078513      addi x10 x15 0
100fc:      01c12403      lw x8 28 x2
10100:      02010113      addi x2 x2 32
10104:      00008067      jalr x0 x1 0

```

a = fp [-20]
b = fp [-28]
c = fp [-24]

a1 = b
a0 = c
jal funcion
c = a0

```

1 int funcion(int,int);
2
3 int main(){
4     int a=8;
5     int b=10;
6     int c=0;
7
8     while (a>0){
9         c=funcion(c,b);
10        a--;
11    }
12
13    return c;
14 }
15
16 int funcion(int a,int b){
17     return a+b;
18 }
19

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00112e23      sw x1 28 x2
1007c:      00812c23      sw x8 24 x2
10080:      02010413      addi x8 x2 32
10084:      00800793      addi x15 x0 8
10088:      fef42623      sw x15 -20 x8
1008c:      00a00793      addi x15 x0 10
10090:      fef42223      sw x15 -28 x8
10094:      fe042423      sw x0 -24 x8
10098:      0200006f      jal x0 32
1009c:      fe442583      lw x11 -28 x8
100a0:      fe842503      lw x10 -24 x8
100a4:      034000ef      jal x1 52 <funcion>
100a8:      fea42423      sw x10 -24 x8
100ac:      fec42783      lw x15 -20 x8
100b0:      fff78793      addi x15 x15 -1
100b4:      fef42623      sw x15 -20 x8
100b8:      fec42783      lw x15 -20 x8
100bc:      fef040e3      blt x0 x15 -32
100c0:      fe842783      lw x15 -24 x8
100c4:      00078513      addi x10 x15 0
100c8:      01c12083      lw x1 28 x2
100cc:      01812403      lw x8 24 x2
100d0:      02010113      addi x2 x2 32
100d4:      00008067      jalr x0 x1 0

000100d8 <funcion>:
100d8:      fe010113      addi x2 x2 -32
100dc:      00812e23      sw x8 28 x2
100e0:      02010413      addi x8 x2 32
100e4:      fea42623      sw x10 -20 x8
100e8:      feb42423      sw x11 -24 x8
100ec:      fec42703      lw x14 -20 x8
100f0:      fe842783      lw x15 -24 x8
100f4:      00f707b3      add x15 x14 x15
100f8:      00078513      addi x10 x15 0
100fc:      01c12403      lw x8 28 x2
10100:      02010113      addi x2 x2 32
10104:      00008067      jalr x0 x1 0

```

a = fp [-20]
b = fp [-28]
c = fp [-24]

cada función tiene su espacio de stack y frame pointer

se restaura cuando a los valores originales antes de retornar

```

1 int funcion(int,int);
2
3 int main(){
4     int a=8;
5     int b=10;
6     int c=0;
7
8     while (a>0){
9         c=funcion(c,b);
10        a--;
11    }
12
13    return c;
14 }
15
16 int funcion(int a,int b){
17     return a+b;
18 }
19

```

```

00010074 <main>:
10074:      fe010113      addi x2,x2,-32
10078:      00112e23      sw x1,28(x2)
1007c:      00812c23      sw x8,24(x2)
10080:      02010413      addi x8,x2,32
10084:      00800793      addi x15,x0,8
10088:      fef42623      sw x15,-20(x8)
1008c:      00a00793      addi x15,x0,10
10090:      fef42223      sw x15,-28(x8)
10094:      fe042423      sw x0,-24(x8)
10098:      0200006f      jal x0,32
1009c:      fe442583      lw x11,-28(x8)
100a0:      fe842503      lw x10,-24(x8)
100a4:      034000ef      jal x1,52(<funcion>)
100a8:      fea42423      sw x10,-24(x8)
100ac:      fec42783      lw x15,-20(x8)
100b0:      fff78793      addi x15,x15,-1
100b4:      fef42623      sw x15,-20(x8)
100b8:      fec42783      lw x15,-20(x8)
100bc:      fef040e3      blt x0,x15,-32
100c0:      fe842783      lw x15,-24(x8)
100c4:      00078513      addi x10,x15,0
100c8:      01c12083      lw x1,28(x2)
100cc:      01812403      lw x8,24(x2)
100d0:      02010113      addi x2,x2,32
100d4:      00008067      jalr x0,x1,0

```

```

000100d8 <funcion>:
100d8:      fe010113      addi x2,x2,-32
100dc:      00812e23      sw x8,28(x2)
100e0:      02010413      addi x8,x2,32
100e4:      fea42623      sw x10,-20(x8)
100e8:      feb42423      sw x11,-24(x8)
100ec:      fec42703      lw x14,-20(x8)
100f0:      fe842783      lw x15,-24(x8)
100f4:      00f707b3      add x15,x14,x15
100f8:      00078513      addi x10,x15,0
100fc:      01c12403      lw x8,28(x2)
10100:      02010113      addi x2,x2,32
10104:      00008067      jalr x0,x1,0

```

a = fp [-20]
b = fp [-28]
c = fp [-24]

a' = fp [-20]
b' = fp [-24]

```

1 int funcion(int,int);
2
3 int main(){
4     int a=8;
5     int b=10;
6     int c=0;
7
8     while (a>0){
9         c=funcion(c,b);
10        a--;
11    }
12
13    return c;
14 }
15
16 int funcion(int a,int b){
17     return a+b;
18 }
19

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00112e23      sw x1 28 x2
1007c:      00812c23      sw x8 24 x2
10080:      02010413      addi x8 x2 32
10084:      00800793      addi x15 x0 8
10088:      fef42623      sw x15 -20 x8
1008c:      00a00793      addi x15 x0 10
10090:      fef42223      sw x15 -28 x8
10094:      fe042423      sw x0 -24 x8
10098:      0200006f      jal x0 32
1009c:      fe442583      lw x11 -28 x8
100a0:      fe842503      lw x10 -24 x8
100a4:      034000ef      jal x1 52 <funcion>
100a8:      fea42423      sw x10 -24 x8
100ac:      fec42783      lw x15 -20 x8
100b0:      fff78793      addi x15 x15 -1
100b4:      fef42623      sw x15 -20 x8
100b8:      fec42783      lw x15 -20 x8
100bc:      fef040e3      blt x0 x15 -32
100c0:      fe842783      lw x15 -24 x8
100c4:      00078513      addi x10 x15 0
100c8:      01c12083      lw x1 28 x2
100cc:      01812403      lw x8 24 x2
100d0:      02010113      addi x2 x2 32
100d4:      00008067      jalr x0 x1 0

```

```

000100d8 <funcion>:
100d8:      fe010113      addi x2 x2 -32
100dc:      00812e23      sw x8 28 x2
100e0:      02010413      addi x8 x2 32
100e4:      fea42623      sw x10 -20 x8
100e8:      feb42423      sw x11 -24 x8
100ec:      fec42783      lw x14 -20 x8
100f0:      fe842783      lw x15 -24 x8
100f4:      00f707b3      add x15 x14 x15
100f8:      00078513      addi x10 x15 0
100fc:      01c12403      lw x8 28 x2
10100:      02010113      addi x2 x2 32
10104:      00008067      jalr x0 x1 0

```

a = fp [-20]
b = fp [-28]
c = fp [-24]

a' = fp [-20]
b' = fp [-24]

a0 = a'+b'

► Decisión (if)

```

int main(){
    int a=8;
    int b=10;

    int c=0;
    if (a>5){
        c=b+2;
    }else {
        c=b-3;
    }

    return c;
}

```



```

00010074 <main>:
10074:    fe010113    addi x2, x2, -32
10078:    00812e23    sw x8, 28(x2)
1007c:    02010413    addi x8, x2, 32
10080:    00800793    addi x15, x0, 8
10084:    fef42423    sw x15, -24(x8)
10088:    00a00793    addi x15, x0, 10
1008c:    fef42223    sw x15, -28(x8)
10090:    fe042623    sw x0, -20(x8)
10094:    fe842703    lw x14, -24(x8)
10098:    00500793    addi x15, x0, 5
1009c:    00e7da63    bge x15, x14, 20
100a0:    fe442783    lw x15, -28(x8)
100a4:    00278793    addi x15, x15, 2
100a8:    fef42623    sw x15, -20(x8)
100ac:    0100006f    jal x0, 16
100b0:    fe442783    lw x15, -28(x8)
100b4:    ffd78793    addi x15, x15, -3
100b8:    fef42623    sw x15, -20(x8)
100bc:    fec42783    lw x15, -20(x8)
100c0:    00078513    addi x10, x15, 0
100c4:    01c12403    lw x8, 28(x2)
100c8:    02010113    addi x2, x2, 32
100cc:    00008067    jalr x0, x1, 0

```

a = fp [-24]
b = fp [-28]
c = fp [-20]

```

int main(){
    int a=8;
    int b=10;

    int c=0;
    if (a>5){
        c=b+2;
    }else {
        c=b-3;
    }

    return c;
}

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00812e23      sw x8 28 x2
1007c:      02010413      addi x8 x2 32
10080:      00800793      addi x15 x0 8
10084:      fef42423      sw x15 -24 x8
10088:      00a00793      addi x15 x0 10
1008c:      fef42223      sw x15 -28 x8
10090:      fe042623      sw x0 -20 x8
10094:      fe842703      lw x14 -24 x8
10098:      00500793      addi x15 x0 5
1009c:      00e7da63      bge x15 x14 20
100a0:      fe442783      lw x15 -28 x8
100a4:      00278793      addi x15 x15 2
100a8:      fef42623      sw x15 -20 x8
100ac:      0100006f      jal x0 16
100b0:      fe442783      lw x15 -28 x8
100b4:      ffd78793      addi x15 x15 -3
100b8:      fef42623      sw x15 -20 x8
100bc:      fec42783      lw x15 -20 x8
100c0:      00078513      addi x10 x15 0
100c4:      01c12403      lw x8 28 x2
100c8:      02010113      addi x2 x2 32
100cc:      00008067      jalr x0 x1 0

```

a > 5 true
 a > 5 false

5 >= a ?

a = fp [-24]
 b = fp [-28]
 c = fp [-20]

```

int main(){
    int a=8;
    int b=10;

    int c=0;
    if (a>5){
        c=b+2;
    }else {
        c=b-3;
    }

    return c;
}

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00812e23      sw x8 28 x2
1007c:      02010413      addi x8 x2 32
10080:      00800793      addi x15 x0 8
10084:      fef42423      sw x15 -24 x8
10088:      00a00793      addi x15 x0 10
1008c:      fef42223      sw x15 -28 x8
10090:      fe042623      sw x0 -20 x8
10094:      fe842703      lw x14 -24 x8
10098:      00500793      addi x15 x0 5
1009c:      00e7da63      bge x15 x14 20
100a0:      fe442783      lw x15 -28 x8
100a4:      00278793      addi x15 x15 2
100a8:      fef42623      sw x15 -20 x8
100ac:      0100006f      jal x0 16
100b0:      fe442783      lw x15 -28 x8
100b4:      ffd78793      addi x15 x15 -3
100b8:      fef42623      sw x15 -20 x8
100bc:      fec42783      lw x15 -20 x8
100c0:      00078513      addi x10 x15 0
100c4:      01c12403      lw x8 28 x2
100c8:      02010113      addi x2 x2 32
100cc:      00008067      jalr x0 x1 0

```

a = fp [-24]
b = fp [-28]
c = fp [-20]

a>5 true

```

int main(){
    int a=8;
    int b=10;

    int c=0;
    if (a>5){
        c=b+2;
    }else {
        c=b-3;
    }

    return c;
}

```

```

00010074 <main>:
10074:      fe010113      addi x2 x2 -32
10078:      00812e23      sw x8 28 x2
1007c:      02010413      addi x8 x2 32
10080:      00800793      addi x15 x0 8
10084:      fef42423      sw x15 -24 x8
10088:      00a00793      addi x15 x0 10
1008c:      fef42223      sw x15 -28 x8
10090:      fe042623      sw x0 -20 x8
10094:      fe842703      lw x14 -24 x8
10098:      00500793      addi x15 x0 5
1009c:      00e7da63      bge x15 x14 20
100a0:      fe442783      lw x15 -28 x8
100a4:      00278793      addi x15 x15 2
100a8:      fef42623      sw x15 -20 x8
100ac:      0100006f      jal x0 16
100b0:      fe442783      lw x15 -28 x8
100b4:      ffd78793      addi x15 x15 -3
100b8:      fef42623      sw x15 -20 x8
100bc:      fec42783      lw x15 -20 x8
100c0:      00078513      addi x10 x15 0
100c4:      01c12403      lw x8 28 x2
100c8:      02010113      addi x2 x2 32
100cc:      00008067      jalr x0 x1 0

```

a = fp [-24]
b = fp [-28]
c = fp [-20]

a>5 false

```

int main(){
    int a=8;
    int b=10;

    int c=0;
    if (a>5){
        c=b+2;
    }else {
        c=b-3;
    }

    return c;
}

```

```

00010074 <main>:
10074:    fe010113    addi x2 x2 -32
10078:    00812e23    sw x8 28 x2
1007c:    02010413    addi x8 x2 32
10080:    00800793    addi x15 x0 8
10084:    fef42423    sw x15 -24 x8
10088:    00a00793    addi x15 x0 10
1008c:    fef42223    sw x15 -28 x8
10090:    fe042623    sw x0 -20 x8
10094:    fe842703    lw x14 -24 x8
10098:    00500793    addi x15 x0 5
1009c:    00e7da63    bge x15 x14 20
100a0:    fe442783    lw x15 -28 x8
100a4:    00278793    addi x15 x15 2
100a8:    fef42623    sw x15 -20 x8
100ac:    0100006f    jal x0 16
100b0:    fe442783    lw x15 -28 x8
100b4:    ffd78793    addi x15 x15 -3
100b8:    fef42623    sw x15 -20 x8
100bc:    fec42783    lw x15 -20 x8
100c0:    00078513    addi x10 x15 0
100c4:    01c12403    lw x8 28 x2
100c8:    02010113    addi x2 x2 32
100cc:    00008067    jalr x0 x1 0

```

a = fp [-24]
b = fp [-28]
c = fp [-20]

► Estructuras y punteros

```

1 struct Fecha{
2     unsigned char D;
3     unsigned char M;
4     unsigned short A;
5 };
6 void arreglar(struct Fecha*);
7 int main()
8 {
9     struct Fecha mundial;
10    mundial.D=18;
11    mundial.M=12;
12    mundial.A=2022;
13
14    arreglar( &mundial);
15
16    return mundial.D;
17 }
18
19 void arreglar(struct Fecha* mejico){
20     mejico->D=29;
21     mejico->M=6;
22     mejico->A=1986;
23 }
24

```

```

00010074 <main>:
    10074:    fe010113    addi x2 x2 -32
    10078:    00112e23    sw x1 28 x2
    1007c:    00812c23    sw x8 24 x2
    10080:    02010413    addi x8 x2 32
    10084:    01200793    addi x15 x0 18
    10088:    fef40623    sb x15 -20 x8
    1008c:    00c00793    addi x15 x0 12
    10090:    fef406a3    sb x15 -19 x8
    10094:    7e600793    addi x15 x0 2022
    10098:    fef41723    sh x15 -18 x8
    1009c:    fec40793    addi x15 x8 -20
    100a0:    00078513    addi x10 x15 0
    100a4:    01c000ef    jal x1 28 <arreglar>
    100a8:    fec44783    lbu x15 -20 x8
    100ac:    00078513    addi x10 x15 0
    100b0:    01c12083    lw x1 28 x2
    100b4:    01812403    lw x8 24 x2
    100b8:    02010113    addi x2 x2 32
    100bc:    00008067    jalr x0 x1 0

000100c0 <arreglar>:
    100c0:    fe010113    addi x2 x2 -32
    100c4:    00812e23    sw x8 28 x2
    100c8:    02010413    addi x8 x2 32
    100cc:    fea42623    sw x10 -20 x8
    100d0:    fec42783    lw x15 -20 x8
    100d4:    01d00713    addi x14 x0 29
    100d8:    00e78023    sb x14 0 x15
    100dc:    fec42783    lw x15 -20 x8
    100e0:    00600713    addi x14 x0 6
    100e4:    00e780a3    sb x14 1 x15
    100e8:    fec42783    lw x15 -20 x8
    100ec:    7c200713    addi x14 x0 1986
    100f0:    00e79123    sh x14 2 x15
    100f4:    00000013    addi x0 x0 0
    100f8:    01c12403    lw x8 28 x2
    100fc:    02010113    addi x2 x2 32
    10100:    00008067    jalr x0 x1 0

```

```

1 struct Fecha{
2     unsigned char D;
3     unsigned char M;
4     unsigned short A;
5 };
6 void arreglar(struct Fecha*);
7 int main()
8 {
9     struct Fecha mundial;
10    mundial.D=18;
11    mundial.M=12;
12    mundial.A=2022;
13
14    arreglar( &mundial);
15
16    return mundial.D;
17 }
18
19 void arreglar(struct Fecha* mejico){
20     mejico->D=29;
21     mejico->M=6;
22     mejico->A=1986;
23 }
24

```

```

00010074 <main>:
10074:    fe010113    addi x2 x2 -32
10078:    00112e23    sw x1 28 x2
1007c:    00812c23    sw x8 24 x2
10080:    02010413    addi x8 x2 32
10084:    01200793    addi x15 x0 18
10088:    fef40623    sb x15 -20 x8
1008c:    00c00793    addi x15 x0 12
10090:    fef406a3    sb x15 -19 x8
10094:    7e600793    addi x15 x0 2022
10098:    fef41723    sh x15 -18 x8
1009c:    fec40793    addi x15 x8 -20
100a0:    00078513    addi x10 x15 0
100a4:    01c000ef    jal x1 28 <arreglar>
100a8:    fec44783    lbu x15 -20 x8
100ac:    00078513    addi x10 x15 0
100b0:    01c12083    lw x1 28 x2
100b4:    01812403    lw x8 24 x2
100b8:    02010113    addi x2 x2 32
100bc:    00008067    jalr x0 x1 0

000100c0 <arreglar>:
100c0:    fe010113    addi x2 x2 -32
100c4:    00812e23    sw x8 28 x2
100c8:    02010413    addi x8 x2 32
100cc:    fea42623    sw x10 -20 x8
100d0:    fec42783    lw x15 -20 x8
100d4:    01d00713    addi x14 x0 29
100d8:    00e78023    sb x14 0 x15
100dc:    fec42783    lw x15 -20 x8
100e0:    00600713    addi x14 x0 6
100e4:    00e780a3    sb x14 1 x15
100e8:    fec42783    lw x15 -20 x8
100ec:    7c200713    addi x14 x0 1986
100f0:    00e79123    sh x14 2 x15
100f4:    00000013    addi x0 x0 0
100f8:    01c12403    lw x8 28 x2
100fc:    02010113    addi x2 x2 32
10100:    00008067    jalr x0 x1 0

```

struct Fecha mundial = fp[-20]

D = fp[-20]

M = fp[-19]

A = fp[-18]

```

1 struct Fecha{
2     unsigned char D;
3     unsigned char M;
4     unsigned short A;
5 };
6 void arreglar(struct Fecha*);
7 int main()
8 {
9     struct Fecha mundial;
10    mundial.D=18;
11    mundial.M=12;
12    mundial.A=2022;
13
14    arreglar( &mundial);
15
16    return mundial.D;
17 }
18
19 void arreglar(struct Fecha* mejico){
20     mejico->D=29;
21     mejico->M=6;
22     mejico->A=1986;
23 }
24

```

```

00010074 <main>:
10074:    fe010113    addi x2 x2 -32
10078:    00112e23    sw x1 28 x2
1007c:    00812c23    sw x8 24 x2
10080:    02010413    addi x8 x2 32
10084:    01200793    addi x15 x0 18
10088:    fef40623    sb x15 -20 x8
1008c:    00c00793    addi x15 x0 12
10090:    fef406a3    sb x15 -19 x8
10094:    7e600793    addi x15 x0 2022
10098:    fef41723    sh x15 -18 x8
1009c:    fec40793    addi x15 x8 -20
100a0:    00078513    addi x10 x15 0
100a4:    01c000ef    jal x1 28 <arreglar>
100a8:    fec44783    lbu x15 -20 x8
100ac:    00078513    addi x10 x15 0
100b0:    01c12083    lw x1 28 x2
100b4:    01812403    lw x8 24 x2
100b8:    02010113    addi x2 x2 32
100bc:    00008067    jalr x0 x1 0

000100c0 <arreglar>:
100c0:    fe010113    addi x2 x2 -32
100c4:    00812e23    sw x8 28 x2
100c8:    02010413    addi x8 x2 32
100cc:    fea42623    sw x10 -20 x8
100d0:    fec42783    lw x15 -20 x8
100d4:    01d00713    addi x14 x0 29
100d8:    00e78023    sb x14 0 x15
100dc:    fec42783    lw x15 -20 x8
100e0:    00600713    addi x14 x0 6
100e4:    00e780a3    sb x14 1 x15
100e8:    fec42783    lw x15 -20 x8
100ec:    7c200713    addi x14 x0 1986
100f0:    00e79123    sh x14 2 x15
100f4:    00000013    addi x0 x0 0
100f8:    01c12403    lw x8 28 x2
100fc:    02010113    addi x2 x2 32
10100:    00008067    jalr x0 x1 0

```

struct Fecha mundial = fp[-20]

D = fp[-20]

M = fp[-19]

A = fp[-18]

x15 = fp - 20 (la dirección)

Recordemos que fp es un registro cuyo contenido apunta a memoria.

Por ende fp-20 es la dirección de memoria donde comienza struct Fecha mundial

Luego a0 = x15 , pasando como argumento &mundial (la dirección donde está almacenada la variable)

```

1 struct Fecha{
2     unsigned char D;
3     unsigned char M;
4     unsigned short A;
5 };
6 void arreglar(struct Fecha*);
7 int main()
8 {
9     struct Fecha mundial;
10    mundial.D=18;
11    mundial.M=12;
12    mundial.A=2022;
13
14    arreglar( &mundial);
15
16    return mundial.D;
17 }
18
19 void arreglar(struct Fecha* mejico){
20     mejico->D=29;
21     mejico->M=6;
22     mejico->A=1986;
23 }
24

```

```

00010074 <main>:
10074:    fe010113    addi x2 x2 -32
10078:    00112e23    sw x1 28 x2
1007c:    00812c23    sw x8 24 x2
10080:    02010413    addi x8 x2 32
10084:    01200793    addi x15 x0 18
10088:    fef40623    sb x15 -20 x8
1008c:    00c00793    addi x15 x0 12
10090:    fef406a3    sb x15 -19 x8
10094:    7e600793    addi x15 x0 2022
10098:    fef41723    sh x15 -18 x8
1009c:    fec40793    addi x15 x8 -20
100a0:    00078513    addi x10 x15 0
100a4:    01c000ef    jal x1 28 <arreglar>
100a8:    fec44783    lbu x15 -20 x8
100ac:    00078513    addi x10 x15 0
100b0:    01c12083    lw x1 28 x2
100b4:    01812403    lw x8 24 x2
100b8:    02010113    addi x2 x2 32
100bc:    00008067    jalr x0 x1 0

```

```

000100c0 <arreglar>:
100c0:    fe010113    addi x2 x2 -32
100c4:    00812e23    sw x8 28 x2
100c8:    02010413    addi x8 x2 32
100cc:    fea42623    sw x10 -20 x8
100d0:    fec42783    lw x15 -20 x8
100d4:    01d00713    addi x14 x0 29
100d8:    00e78023    sb x14 0 x15
100dc:    fec42783    lw x15 -20 x8
100e0:    00600713    addi x14 x0 6
100e4:    00e780a3    sb x14 1 x15
100e8:    fec42783    lw x15 -20 x8
100ec:    7c200713    addi x14 x0 1986
100f0:    00e79123    sh x14 2 x15
100f4:    00000013    addi x0 x0 0
100f8:    01c12403    lw x8 28 x2
100fc:    02010113    addi x2 x2 32
10100:    00008067    jalr x0 x1 0

```

struct Fecha mundial = fp[-20]

D = fp[-20]

M = fp[-19]

A = fp[-18]

**ahora en el fp' la posición fp[-20]
copia la dirección pasada en a0.**

**Quiere decir que ahora:
struct Fecha mejico = fp'[-20]**

```

1 struct Fecha{
2     unsigned char D;
3     unsigned char M;
4     unsigned short A;
5 };
6 void arreglar(struct Fecha*);
7 int main()
8 {
9     struct Fecha mundial;
10    mundial.D=18;
11    mundial.M=12;
12    mundial.A=2022;
13
14    arreglar( &mundial);
15
16    return mundial.D;
17 }
18
19 void arreglar(struct Fecha* mejico){
20     mejico->D=29;
21     mejico->M=6;
22     mejico->A=1986;
23 }
24

```

```

00010074 <main>:
10074:    fe010113    addi x2 x2 -32
10078:    00112e23    sw x1 28 x2
1007c:    00812c23    sw x8 24 x2
10080:    02010413    addi x8 x2 32
10084:    01200793    addi x15 x0 18
10088:    fef40623    sb x15 -20 x8
1008c:    00c00793    addi x15 x0 12
10090:    fef406a3    sb x15 -19 x8
10094:    7e600793    addi x15 x0 2022
10098:    fef41723    sh x15 -18 x8
1009c:    fec40793    addi x15 x8 -20
100a0:    00078513    addi x10 x15 0
100a4:    01c000ef    jal x1 28 <arreglar>
100a8:    fec44783    lbu x15 -20 x8
100ac:    00078513    addi x10 x15 0
100b0:    01c12083    lw x1 28 x2
100b4:    01812403    lw x8 24 x2
100b8:    02010113    addi x2 x2 32
100bc:    00008067    jalr x0 x1 0

```

```

000100c0 <arreglar>:
100c0:    fe010113    addi x2 x2 -32
100c4:    00812e23    sw x8 28 x2
100c8:    02010413    addi x8 x2 32
100cc:    fea42623    sw x10 -20 x8
100d0:    fec42783    lw x15 -20 x8
100d4:    01d00713    addi x14 x0 29
100d8:    00e78023    sb x14 0 x15
100dc:    fec42783    lw x15 -20 x8
100e0:    00600713    addi x14 x0 6
100e4:    00e780a3    sb x14 1 x15
100e8:    fec42783    lw x15 -20 x8
100ec:    7c200713    addi x14 x0 1986
100f0:    00e79123    sh x14 2 x15
100f4:    00000013    addi x0 x0 0
100f8:    01c12403    lw x8 28 x2
100fc:    02010113    addi x2 x2 32
10100:    00008067    jalr x0 x1 0

```

struct Fecha mundial = fp[-20]

D = fp[-20]

M = fp[-19]

A = fp[-18]

**Ahora la base de la estructura está en fp'[-20]
Copia esta dirección a x15 y luego almacena:**

D = 29 en 0(x15)

M = 6 en 1(x15)

A = 1986 en 2(x15)

**Recordemos que x15 apunta a la posición real
de donde se encuentra struct Fecha mundial.
No se pasaron datos por copia (valor) sino por
referencia**

► Optimizaciones

```

1
2 struct Fecha{
3     unsigned char D ;
4     unsigned char M ;
5     unsigned short A ;
6 } __attribute__((packed)) ;
7
8 int main()
9 {
10     volatile struct Fecha mundial;
11     mundial.D=18;
12     mundial.M=12;
13     mundial.A=2022;
14     return mundial.D;
15 }
16
17

```

00010074 <main>:

10074:	ff010113	addi x2 x2 -16
10078:	01200793	addi x15 x0 18
1007c:	00f10623	sb x15 12 x2
10080:	00c00793	addi x15 x0 12
10084:	00f106a3	sb x15 13 x2
10088:	7e600793	addi x15 x0 2022
1008c:	00f11723	sh x15 14 x2
10090:	00c14503	lbu x10 12 x2
10094:	01010113	addi x2 x2 16
10098:	00008067	jalr x0 x1 0

Settings

Environment
 Simulator
Compiler
 Editor

RISC-V C/C++ Compiler

Providing a compatible RISC-V C/C++ con-
 execution of C-language programs within

A compiler may be autodetected if availal

Compiler path:

Compiler arguments:

Linker arguments:

```

1
2 struct Fecha{
3     unsigned char D ;
4     unsigned char M ;
5     unsigned short A ;
6 } __attribute__((packed)) ;
7
8 int main()
9 {
10     volatile struct Fecha mundial;
11     mundial.D=18;
12     mundial.M=12;
13     mundial.A=2022;
14     return mundial.D;
15 }
16
17

```

00010074 <main>:

10074:	ff010113	addi x2 x2 -16
10078:	01200793	addi x15 x0 18
1007c:	00f10623	sb x15 12 x2
10080:	00c00793	addi x15 x0 12
10084:	00f106a3	sb x15 13 x2
10088:	7e600793	addi x15 x0 2022
1008c:	00f11723	sh x15 14 x2
10090:	00c14503	lbu x10 12 x2
10094:	01010113	addi x2 x2 16
10098:	00008067	jalr x0 x1 0

Settings

Environment
 Simulator
 Compiler
 Editor

RISC-V C/C++ Compiler

Providing a compatible RISC-V C/C++ con-
 execution of C-language programs within

A compiler may be autodetected if availal

Compiler path: wipeme/toolchain/rv32i/

Compiler arguments: -O1 -nostdlib

Linker arguments: -static -lm

```

1 struct Fecha{
2     unsigned char D ;
3     unsigned char M ;
4     unsigned short A ;
5 } __attribute__((packed)) ;
6
7
8 int main()
9 {
10     volatile struct Fecha mundial;
11     mundial.D=18;
12     mundial.M=12;
13     mundial.A=2022;
14     return mundial.D;
15 }
16
17

```

Vemos que la estructura se almacena en:

D = sp[12] (8 bits)

M = sp[13] (8 bits)

A = sp[14] (16 bits)

sizeof(struct Fecha) es 4 bytes

```

00010074 <main>:
10074:    ff010113    addi x2, x2, -16
10078:    01200793    addi x15, x0, 18
1007c:    00f10623    sb x15, 12(x2)
10080:    00c00793    addi x15, x0, 12
10084:    00f106a3    sb x15, 13(x2)
10088:    7e600793    addi x15, x0, 2022
1008c:    00f11723    sh x15, 14(x2)
10090:    00c14503    lbu x10, 12(x2)
10094:    01010113    addi x2, x2, 16
10098:    00008067    jalr x0, x1, 0

```

Settings

Environment
 Simulator
Compiler
 Editor

RISC-V C/C++ Compiler

Providing a compatible RISC-V C/C++ con-
 execution of C-language programs within

A compiler may be autodetected if availal

Compiler path:

Compiler arguments:

Linker arguments:

Sin optimizar -O0

```

1 int main(){
2     int a=8;
3     int b=10;
4     int c=0;
5     while(b>0){
6         c=c+a;
7         b--;
8     }
9     return c;
10 }
11 |

```

```

00010074 <main>:
10074:      fe010113      addi x2, x2, -32
10078:      00812e23      sw x8, 28(x2)
1007c:      02010413      addi x8, x2, 32
10080:      00800793      addi x15, x0, 8
10084:      fef42223      sw x15, -28(x8)
10088:      00a00793      addi x15, x0, 10
1008c:      fef42623      sw x15, -20(x8)
10090:      fe042423      sw x0, -24(x8)
10094:      0200006f      jal x0, 32
10098:      fe842703      lw x14, -24(x8)
1009c:      fe442783      lw x15, -28(x8)
100a0:      00f707b3      add x15, x14, x15
100a4:      fef42423      sw x15, -24(x8)
100a8:      fec42783      lw x15, -20(x8)
100ac:      fff78793      addi x15, x15, -1
100b0:      fef42623      sw x15, -20(x8)
100b4:      fec42783      lw x15, -20(x8)
100b8:      fef040e3      blt x0, x15, -32
100bc:      fe842783      lw x15, -24(x8)
100c0:      00078513      addi x10, x15, 0
100c4:      01c12403      lw x8, 28(x2)
100c8:      02010113      addi x2, x2, 32
100cc:      00008067      jalr x0, x1, 0

```

Optimizar nivel 1 (mínimo) -O1

```

1 int main(){
2     int a=8;
3     int b=10;
4     int c=0;
5     while(b>0){
6         c=c+a;
7         b--;
8     }
9     return c;
10 }

```

```

00010074 <main>:
10074:      05000513      addi x10, x0, 80
10078:      00008067      jalr x0, x1, 0

```

► Vectores

```

1
2
3 int main()
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
7
8     int a=10;
9
10    a=vector[i];
11    return a;
12 }
13
14 |

```

```

00010074 <main>:
10074:      fd010113      addi x2 x2 -48
10078:      02812623      sw x8 44 x2
1007c:      03010413      addi x8 x2 48
10080:      000107b7      lui x15 0x10
10084:      0ec78793      addi x15 x15 236
10088:      0007a583      lw x11 0 x15
1008c:      0047a603      lw x12 4 x15
10090:      0087a683      lw x13 8 x15
10094:      00c7a703      lw x14 12 x15
10098:      0107a783      lw x15 16 x15
1009c:      fcb42a23      sw x11 -44 x8
100a0:      fcc42c23      sw x12 -40 x8
100a4:      fcd42e23      sw x13 -36 x8
100a8:      fee42023      sw x14 -32 x8
100ac:      fef42223      sw x15 -28 x8
100b0:      00300793      addi x15 x0 3
100b4:      fef42623      sw x15 -20 x8
100b8:      00a00793      addi x15 x0 10
100bc:      fef42423      sw x15 -24 x8
100c0:      fec42783      lw x15 -20 x8
100c4:      00279793      slli x15 x15 2
100c8:      ff078793      addi x15 x15 -16
100cc:      008787b3      add x15 x15 x8
100d0:      fe47a783      lw x15 -28 x15
100d4:      fef42423      sw x15 -24 x8
100d8:      fe842783      lw x15 -24 x8
100dc:      00078513      addi x10 x15 0
100e0:      02c12403      lw x8 44 x2
100e4:      03010113      addi x2 x2 48
100e8:      00008067      jalr x0 x1 0

```

?

```

1
2
3 int main()
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
7
8     int a=10;
9
10    a=vector[i];
11    return a;
12 }
13
14 |

```

00010074 <main>:

10074:	fd010113	addi x2 x2 -48
10078:	02812623	sw x8 44 x2
1007c:	03010413	addi x8 x2 48
10080:	000107b7	lui x15 0x10
10084:	0ec78793	addi x15 x15 236
10088:	0007a583	lw x11 0 x15
1008c:	0047a603	lw x12 4 x15
10090:	0087a683	lw x13 8 x15
10094:	00c7a703	lw x14 12 x15
10098:	0107a783	lw x15 16 x15
1009c:	fcba42a23	sw x11 -44 x8
100a0:	fcc42c23	sw x12 -40 x8
100a4:	fcd42e23	sw x13 -36 x8
100a8:	fee42023	sw x14 -32 x8
100ac:	fef42223	sw x15 -28 x8
100b0:	00300793	addi x15 x0 3
100b4:	fef42623	sw x15 -20 x8
100b8:	00a00793	addi x15 x0 10
100bc:	fef42423	sw x15 -24 x8
100c0:	fec42783	lw x15 -20 x8
100c4:	00279793	slli x15 x15 2
100c8:	ff078793	addi x15 x15 -16
100cc:	008787b3	add x15 x15 x8
100d0:	fe47a783	lw x15 -28 x15
100d4:	fef42423	sw x15 -24 x8
100d8:	fe842783	lw x15 -24 x8
100dc:	00078513	addi x10 x15 0
100e0:	02c12403	lw x8 44 x2
100e4:	03010113	addi x2 x2 48
100e8:	00008067	jalr x0 x1 0

```
1
2
3 int main()
```

```
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
```

```
7
8     int a=10;
```

```
9
10    a=vector[i];
11    return a;
```

```
12 }
13
14 |
```

Memory map

Name	Size	Range
.symtab	272	0x00000000 - 0x0000010f
.text	120	0x00010074 - 0x000100eb
.rodata	20	0x000100ec - 0x000100ff

0x000100fc	500	244	1	0	0
0x000100f8	400	144	1	0	0
0x000100f4	300	44	1	0	0
0x000100f0	200	200	0	0	0
0x000100ec	100	100	0	0	0

```

100b0: 00300793 addi x15 x0 3
100b4: fef42623 sw x15 -20 x8
100b8: 00a00793 addi x15 x0 10
100bc: fef42423 sw x15 -24 x8
100c0: fec42783 lw x15 -20 x8
100c4: 00279793 slli x15 x15 2
100c8: ff078793 addi x15 x15 -16
100cc: 008787b3 add x15 x15 x8
100d0: fe47a783 lw x15 -28 x15
100d4: fef42423 sw x15 -24 x8
100d8: fe842783 lw x15 -24 x8
100dc: 00078513 addi x10 x15 0
100e0: 02c12403 lw x8 44 x2
100e4: 03010113 addi x2 x2 48
100e8: 00008067 jalr x0 x1 0

```

0x100ec = 100

$236_{10} = ec_{16}$

x15 = 0x100ec

```
1
2
3 int main()
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
7
8     int a=10;
9
10    a=vector[i];
11    return a;
12 }
13
14 |
```

```
00010074 <main>:
10074:    fd010113    addi x2 x2 -48
10078:    02812623    sw x8 44 x2
1007c:    03010413    addi x8 x2 48
10080:    000107b7    lui x15 0x10
10084:    0ec78793    addi x15 x15 236
10088:    0007a583    lw x11 0 x15
1008c:    0047a603    lw x12 4 x15
10090:    0087a683    lw x13 8 x15
10094:    00c7a703    lw x14 12 x15
10098:    0107a783    lw x15 16 x15
1009c:    fcb42a23    sw x11 -44 x8
100a0:    fcc42c23    sw x12 -40 x8
100a4:    fcd42e23    sw x13 -36 x8
100a8:    fee42023    sw x14 -32 x8
100ac:    fef42223    sw x15 -28 x8
100b0:    00300793    addi x15 x0 3
100b4:    fef42623    sw x15 -20 x8
100b8:    00a00793    addi x15 x0 10
100bc:    fef42423    sw x15 -24 x8
100c0:    fec42783    lw x15 -20 x8
100c4:    00279793    slli x15 x15 2
100c8:    ff078793    addi x15 x15 -16
100cc:    008787b3    add x15 x15 x8
100d0:    fe47a783    lw x15 -28 x15
100d4:    fef42423    sw x15 -24 x8
100d8:    fe842783    lw x15 -24 x8
100dc:    00078513    addi x10 x15 0
100e0:    02c12403    lw x8 44 x2
100e4:    03010113    addi x2 x2 48
100e8:    00008067    jalr x0 x1 0
```

```

1
2
3 int main()
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
7
8     int a=10;
9
10    a=vector[i];
11    return a;
12 }
13
14 |

```

00010074 <main>:

10074:	fd010113	addi x2 x2 -48
10078:	02812623	sw x8 44 x2
1007c:	03010413	addi x8 x2 48
10080:	000107b7	lui x15 0x10
10084:	0ec78793	addi x15 x15 236
10088:	0007a583	lw x11 0 x15
1008c:	0047a603	lw x12 4 x15
10090:	0087a683	lw x13 8 x15
10094:	00c7a703	lw x14 12 x15
10098:	0107a783	lw x15 16 x15
1009c:	fc42a23	sw x11 -44 x8
100a0:	fcc42c23	sw x12 -40 x8
100a4:	fcd42e23	sw x13 -36 x8
100a8:	fee42023	sw x14 -32 x8
100ac:	fef42223	sw x15 -28 x8
100b0:	00300793	addi x15 x0 3
100b4:	fef42623	sw x15 -20 x8
100b8:	00a00793	addi x15 x0 10
100bc:	fef42423	sw x15 -24 x8
100c0:	fec42783	lw x15 -20 x8
100c4:	00279793	slli x15 x15 2
100c8:	ff078793	addi x15 x15 -16
100cc:	008787b3	add x15 x15 x8
100d0:	fe47a783	lw x15 -28 x15
100d4:	fef42423	sw x15 -24 x8
100d8:	fe842783	lw x15 -24 x8
100dc:	00078513	addi x10 x15 0
100e0:	02c12403	lw x8 44 x2
100e4:	03010113	addi x2 x2 48
100e8:	00008067	jalr x0 x1 0

0x100ec = 100

$236_{10} = ec_{16}$

x15 = 0x100ec

**Copia cada
elemento del vector
a su espacio de
variables locales
RAM**

**Notemos que fp[-32]
contiene el 400.**

```

1
2
3 int main()
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
7
8     int a=10;
9
10    a=vector[i];
11    return a;
12 }
13
14 |

```

00010074 <main>:

10074:	fd010113	addi x2 x2 -48
10078:	02812623	sw x8 44 x2
1007c:	03010413	addi x8 x2 48
10080:	000107b7	lui x15 0x10
10084:	0ec78793	addi x15 x15 236
10088:	0007a583	lw x11 0 x15
1008c:	0047a603	lw x12 4 x15
10090:	0087a683	lw x13 8 x15
10094:	00c7a703	lw x14 12 x15
10098:	0107a783	lw x15 16 x15
1009c:	fc42a23	sw x11 -44 x8
100a0:	fcc42c23	sw x12 -40 x8
100a4:	fcd42e23	sw x13 -36 x8
100a8:	fee42023	sw x14 -32 x8
100ac:	fef42223	sw x15 -28 x8
100b0:	00300793	addi x15 x0 3
100b4:	fef42623	sw x15 -20 x8
100b8:	00a00793	addi x15 x0 10
100bc:	fef42423	sw x15 -24 x8
100c0:	fec42783	lw x15 -20 x8
100c4:	00279793	slli x15 x15 2
100c8:	ff078793	addi x15 x15 -16
100cc:	008787b3	add x15 x15 x8
100d0:	fe47a783	lw x15 -28 x15
100d4:	fef42423	sw x15 -24 x8
100d8:	fe842783	lw x15 -24 x8
100dc:	00078513	addi x10 x15 0
100e0:	02c12403	lw x8 44 x2
100e4:	03010113	addi x2 x2 48
100e8:	00008067	jalr x0 x1 0

0x100ec = 100

$236_{10} = ec_{16}$

x15 = 0x100ec

i = fp[-20] = 3
a = fp[-24] = 10

```

1
2
3 int main()
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
7
8     int a=10;
9
10    a=vector[i];
11    return a;
12 }
13
14 |

```

00010074 <main>:

10074:	fd010113	addi x2 x2 -48
10078:	02812623	sw x8 44 x2
1007c:	03010413	addi x8 x2 48
10080:	000107b7	lui x15 0x10
10084:	0ec78793	addi x15 x15 236
10088:	0007a583	lw x11 0 x15
1008c:	0047a603	lw x12 4 x15
10090:	0087a683	lw x13 8 x15
10094:	00c7a703	lw x14 12 x15
10098:	0107a783	lw x15 16 x15
1009c:	fc42a23	sw x11 -44 x8
100a0:	fcc42c23	sw x12 -40 x8
100a4:	fcd42e23	sw x13 -36 x8
100a8:	fee42023	sw x14 -32 x8
100ac:	fef42223	sw x15 -28 x8
100b0:	00300793	addi x15 x0 3
100b4:	fef42623	sw x15 -20 x8
100b8:	00a00793	addi x15 x0 10
100bc:	fef42423	sw x15 -24 x8
100c0:	fec42783	lw x15 -20 x8
100c4:	00279793	slli x15 x15 2
100c8:	ff078793	addi x15 x15 -16
100cc:	008787b3	add x15 x15 x8
100d0:	fe47a783	lw x15 -28 x15
100d4:	fef42423	sw x15 -24 x8
100d8:	fe842783	lw x15 -24 x8
100dc:	00078513	addi x10 x15 0
100e0:	02c12403	lw x8 44 x2
100e4:	03010113	addi x2 x2 48
100e8:	00008067	jalr x0 x1 0

0x100ec = 100

$236_{10} = ec_{16}$

x15 = 0x100ec

x15 = i = 3

x15 = x15*4 (12)

x15 = x15-16 (-4)

x15 = fp - 4

x15 = fp-4-28 (32)

```

1
2
3 int main()
4 {
5     int vector[] = {100,200,300,400,500};
6     int i=3;
7
8     int a=10;
9
10    a=vector[i];
11    return a;
12 }
13
14 |

```

00010074 <main>:

10074:	fd010113	addi x2 x2 -48
10078:	02812623	sw x8 44 x2
1007c:	03010413	addi x8 x2 48
10080:	000107b7	lui x15 0x10
10084:	0ec78793	addi x15 x15 236
10088:	0007a583	lw x11 0 x15
1008c:	0047a603	lw x12 4 x15
10090:	0087a683	lw x13 8 x15
10094:	00c7a703	lw x14 12 x15
10098:	0107a783	lw x15 16 x15
1009c:	fc42a23	sw x11 -44 x8
100a0:	fcc42c23	sw x12 -40 x8
100a4:	fcd42e23	sw x13 -36 x8
100a8:	fee42023	sw x14 -32 x8
100ac:	fef42223	sw x15 -28 x8
100b0:	00300793	addi x15 x0 3
100b4:	fef42623	sw x15 -20 x8
100b8:	00a00793	addi x15 x0 10
100bc:	fef42423	sw x15 -24 x8
100c0:	fec42783	lw x15 -20 x8
100c4:	00279793	slli x15 x15 2
100c8:	ff078793	addi x15 x15 -16
100cc:	008787b3	add x15 x15 x8
100d0:	fe47a783	lw x15 -28 x15
100d4:	fef42423	sw x15 -24 x8
100d8:	fe842783	lw x15 -24 x8
100dc:	00078513	addi x10 x15 0
100e0:	02c12403	lw x8 44 x2
100e4:	03010113	addi x2 x2 48
100e8:	00008067	jalr x0 x1 0

0x100ec = 100

$236_{10} = ec_{16}$

x15 = 0x100ec

x15 = i = 3

x15 = x15*4 (12)

x15 = x15-16 (-4)

x15 = fp - 4

x15 = fp-4-28 (32)

0x100ec = 100

$236_{10} = ec_{16}$

x15 = 0x100ec

a = 400

**a0 = a
return a0**

00010074 <main>:

10074:	fd010113	addi x2 x2 -48
10078:	02812623	sw x8 44 x2
1007c:	03010413	addi x8 x2 48
10080:	000107b7	lui x15 0x10
10084:	0ec78793	addi x15 x15 236
10088:	0007a583	lw x11 0 x15
1008c:	0047a603	lw x12 4 x15
10090:	0087a683	lw x13 8 x15
10094:	00c7a703	lw x14 12 x15
10098:	0107a783	lw x15 16 x15
1009c:	fc42a23	sw x11 -44 x8
100a0:	fcc42c23	sw x12 -40 x8
100a4:	fcd42e23	sw x13 -36 x8
100a8:	fee42023	sw x14 -32 x8
100ac:	fef42223	sw x15 -28 x8
100b0:	00300793	addi x15 x0 3
100b4:	fef42623	sw x15 -20 x8
100b8:	00a00793	addi x15 x0 10
100bc:	fef42423	sw x15 -24 x8
100c0:	fec42783	lw x15 -20 x8
100c4:	00279793	slli x15 x15 2
100c8:	ff078793	addi x15 x15 -16
100cc:	008787b3	add x15 x15 x8
100d0:	fe47a783	lw x15 -28 x15
100d4:	fef42423	sw x15 -24 x8
100d8:	fe842783	lw x15 -24 x8
100dc:	00078513	addi x10 x15 0
100e0:	02c12403	lw x8 44 x2
100e4:	03010113	addi x2 x2 48
100e8:	00008067	jalr x0 x1 0